



Ministry of Higher Education and Scientific Research
University of Technology
College of Computer Engineering
Computer Network Engineering & Cyber Security
Department



Design And Implementation of an Efficient and Secure Examination Management Framework

Graduation project submitted to the College of Computer
Engineering, in partial fulfillment of the requirements for the
(B.Sc.) degree in Computer Engineering.

By

Al-hassan Oday Sahib

Noor Hassan Abdulmahdi

Supervised by

Prof. Dhari A. Mahmood

2025-2026

Supervisor's Certification

I certify that the preparation of this project entitled ([Design And Implementation of an Efficient and Secure Examination Management Framework](#)) was prepared by ([Al-hassan Oday Sahib & Noor Hassan Abdulmahdi](#)) under my supervision at the Computer Engineering Department, College of Computer Engineering, the University of Technology in partial fulfillment of the requirements for the degree of B.Sc. in Computer Engineering.

Supervisor's signature:

Supervisor's name:

Supervisor's Scientific Degree:

Date:

Examination Committee Certification

We certify that we have read this project entitled ([Design And Implementation of an Efficient and Secure Examination Management Framework](#)) , and as an examining committee examined the students ([Al-hassan Oday Sahib & Noor Hassan Abdulmahdi](#)) in its contents, and our opinion, it meets the standards of the B.Sc. Computer Engineering.

Signature:

Name:

Scientific Degree:

Date:

(Member)

Signature:

Name:

Scientific Degree:

Date:

(Member)

Signature:

Name:

Scientific Degree:

Date:

(Member)

Signature:

Name:

Scientific Degree:

Date:

(Chairman)

Signature:

Name:

Scientific Degree:

Date:

Head of Computer Engineering Department

Acknowledgements

We would like to express our sincere gratitude and profound appreciation to our supervisor, **Prof. Dhari A. Mahmood**, for his invaluable guidance, continuous support, and insightful feedback throughout the development of this project. His expertise and encouragement were instrumental in shaping the direction of this research.

Our thanks also extend to the **Computer Engineering Department** at the **University of Technology** for providing the academic environment and resources necessary for this study. We are grateful to all the professors and staff who have shared their knowledge and experience with us during our years of study.

Finally, we owe a debt of gratitude to our families for their unconditional love, patience, and unwavering support. This achievement would not have been possible without their constant encouragement and belief in our potential.

Abstract

The reliance on digital infrastructure in academic institutions is rapidly increasing. The need for a **robust, secure, scalable, and performant examination system** is inevitable. This thesis tries to represent a design and implementation to solve such a case as a *web-based examination management system application*, keeping in mind all the mentioned aspects, in addition to a clean and modern architecture, a clean and easy interface, easy to scale and deploy with modern tooling, flexibility of question symbols and formats, and a role-aware digital platform. aimed to replace the manual process of traditional examinations.

The system supports three distinct roles (*Administrator, Instructor, and Student*), each with its own set of capabilities and privileges, utilizing role-based access control as an **HTTP middleware** and **database constraints**. The administrator is responsible for managing exams, subjects, user assignments, and deletions through a dedicated dashboard. As for the instructors, they are responsible for the creation of the question in the form of an **MCQ question** with three types of questions: **TEXT**: which can deal with mathematical notations using LaTeX directly in the browser. **CODE**, which can highlight a code snippet according to the written programming language. **IMAGE**: This is the final type that can add an image to a question. As for the Student role, after the admin enrolls the student successfully, they can simply enter the exam or the exam collection and submit before the end of the time duration; otherwise, the attempt will be auto-submitted, and they will receive automatically graded scores upon completion that can be downloaded as a **PDF format**.

This framework is primarily built in the **Go programming language** as its backend language, using a clean architecture pattern known as *Hexagonal Architecture (Port and Adapters)*, a *domain-driven architecture* that focuses on the separation of concerns and dependency decoupling. As for the frontend it is rendered using a type-safe compiled go template engine called **Templ**, with a *lightweight* interactivity framework with **Ajax (Asynchronous JavaScript And XML)** capabilities and minimal *javascript* code. Data persistence is handled by **PostgreSQL**, with database integrity enforced through custom triggers, constraints, and stored procedures.

Security measures embedded in the system include **JWT-based cookie authentication**, *role-based access control*, **CSRF** protection, **bcrypt** password hashing, **TLS encryption**, HTTP security headers, and comprehensive structured logging using **Uber's Zap** library. The architecture prioritizes correctness and data integrity at every layer, from the application service down to the relational schema.

Keywords: Go, PostgreSQL, Templ, HTMX, Clean Architecture, Web Security, Examination Management System

Table of Contents

1 Introduction	1
1.1 Overview	2
1.2 Problem Statement	2
1.3 Aim of the Project	3
1.4 Scope	3
1.5 Objective	3
1.6 Organization of the Project	4
2 Literature Review	5
2.1 E-Assessment Systems	6
2.1.1 The advantages of such a platform	6
2.2 Evolution of E-Assessment Systems	6
2.3 Static sites vs. Dynamic sites	8
2.3.1 Static sites	8
2.3.2 Dynamic sites	8
2.4 Type-Safe Templating in Web Applications	9
2.4.1 Traditional Templating Engines	9
2.4.2 Type-Safe Templating Engines	10
2.5 Server-Side Rendering vs. Single-Page Applications	10
2.6 Database Design	13
2.6.1 Database Normalization	13
2.6.2 Database Indexing	14
2.6.3 Database Transaction Management	14
2.7 Compiled queries	16
2.8 Architectural Patterns in Web Back-End Systems	17
2.8.1 MVC (Model-View-Controller)	17
2.8.2 Hexagonal Architecture (Ports and Adapters)	17
2.9 Academic Integrity in Online Examinations	18
2.10 Role-Based Access Control (RBAC)	19
2.11 User Interface	20
2.12 Related Work	20
2.12.1 Moodle	20
2.12.2 Google Forms and Microsoft Forms	21
2.12.3 Existing System (College of Computer Science)	21

3 System Design and

Implementation	22
3.1 Research Methodology	23
3.2 System Design	24
3.2.1 System Architecture Overview	24
3.2.2 Technology Selection and Justification	26
3.2.3 Database Design and Schema	28
3.2.4 User Roles and Access Control Mechanism	30
3.2.5 System Configuration	34
3.2.6 Deployment Strategy	35
3.3 System Implementation	35
3.3.1 The Domain Layer	36
3.3.2 The Port Layer	36
3.3.3 The Infrastructure Layer (Outbound Adapter)	36
3.3.4 The Application Layer	38
3.3.5 The Presentation Layer (Inbound Adapter)	38
3.4 User Interface	39
3.5 Security Mechanisms	40
3.5.1 OWASP Top 10 Security Considerations	40
3.5.2 SQL Injection Prevention Techniques	42
3.5.3 TLS Encryption	42
3.5.4 System Logging and Monitoring	43
3.5.5 Password Security	44
4 Results and Evaluation	46
4.1 System Architecture Evaluation	47
4.1.1 Comparison: Monolithic MVC vs. Hexagonal Architecture	47
4.2 Performance Evaluation	48
4.2.1 Response Time	49
4.2.2 Memory Usage	50
4.3 Comparison with Moodle, and MCQUOT	51
4.4 Solved Problems of the Previous MCQUOT System	51
4.4.1 Database question duplication	51
4.4.2 Database question randomization	52
4.4.3 Question limitation	52
4.4.4 Manual Data Entry	54
5 Conclusion and Future Work	55

5.1	Conclusion	56
5.2	Future Work	56
6	Appendices	57
6.1	Appendix A: Source Code	58
6.1.1	JWT Authentication Middleware	58
6.1.2	Graceful Shutdown	59
6.1.3	Application Transaction Pattern	59
6.1.4	Dependency Injection Pattern (func main)	60
6.1.5	Database functions	62
6.2	Appendix B: User Interface Screens	62
	Bibliography	66

List of Figures

Figure 1	Static Sites Architecture	8
Figure 2	Dynamic Sites Architecture	8
Figure 3	HTMX vs SPA Architecture	11
Figure 4	HTMX Flow Diagram	12
Figure 5	NF1, NF2, NF3 Forms	14
Figure 6	Indexing a database table with a Non-Clustered Index	14
Figure 7	Transaction State Diagram	15
Figure 8	Transaction Manager Diagram	15
Figure 9	MVC Architecture Diagram	17
Figure 10	Hexagonal Architecture Diagram	18
Figure 11	Role-Based Access Control Diagram	19
Figure 12	TemplUI Component Library	20
Figure 13	System Architecture Overview	24
Figure 14	Database Entity-Relationship Schema	29
Figure 15	System Workflow and User Interactions	33
Figure 16	Docker Container Deployment Architecture Workflow	35
Figure 17	Toast Notification After Triggering	40
Figure 18	Bcrypt Password Hashing Algorithm	44
Figure 19	Response time of the system under load using k6 and Matplotlib	49
Figure 20	Memory usage of the system under load using k6	50
Figure 21	Text question type	52
Figure 22	Code highlighted question type	53
Figure 23	Image question type	54
Figure 24	Login Page	62
Figure 25	Admin Dashboard Page	63
Figure 26	Edit Exam Page	63
Figure 27	Edit Subject Dashboard Page	64
Figure 28	All Users Page	64
Figure 29	Student Dashboard Page	65

List of Tables

Table 1		47
Table 2		51



Chapter I

Introduction



1.1. Overview

This project is the design and implementation of a *full-stack web-based application* for an academic examination system. This system is intended to overcome the limitations of the traditional pen and paper examination approach. Also, it is intended to provide solutions for the problems of the existing digital solutions. The system is implemented using a *clean architecture approach* for a scalable and maintainable solution. It uses **Go** on the backend, **PostgreSQL** on the database, and **Templ** and **HTMX** on the frontend. It incorporates a *layered security model* that implements **JWT-based authentication**, **Role-Based Access Control (RBAC)**, **database constraints**, **HTTP Security Headers**, **CSRF tokens**, and **TLS encryption**. Also, support for **LaTeX** math formulas and code blocks. The system is evaluated with a set of benchmarks like performance, resource utilization, and security. Then, the results are compared with other existing solutions.

1.2. Problem Statement

Academic Institutions continue to have challenges in conducting a secure, efficient examination, so the conventional way to do it is to heavily rely on the physical pen and paper approach. which comes with numerous operational problems, high paper and printing costs, physical logistics of distributing and collecting exam booklets, time-consuming manual grading, risk of loss or damage to submitted papers, and limited scalability as the student population grows [1].

Furthermore, the existing digital solution is lacking proper properties and features such as *role-based access control*, *tamper-proof answer recording*, *automated scoring*, *audit trails*, *performance*, and *modern tooling* [2]. Nevertheless, the lack of a good user interface and text formatting constraints.

1.3. Aim of the Project

the primary aim of this project is to design and implement a performant, secure, scalable, maintainable, easy-to-use, and **feature-rich web-based application**, both in the *backend* and *frontend*. this system is intended to serve as a replacement for the manual and slower academic examination workflows.

1.4. Scope

1. Create and Deploy a full-stack web-based application to manage university exams that includes server, database schema, deployment, and server-side rendered frontend.
2. Use Hexagonal Architecture (Port and Adapters) to create a clean architecture design that separates domains and decouples dependencies through abstractions which allows for a testable, scalable, and maintainable system that is capable of easily adapting to future needs.
3. Create a layered security model that implements **JWT-based authentication, Role-Based Access Control (RBAC), database constraints, HTTP Security Headers, CSRF tokens, and TLS encryption**.
4. Leverage **PostgreSQL**'s scripting language and automation through the use of stored functions and triggers to manage data consistency.
5. Reduce database latency through the use of *Database Indexing*.

1.5. Objective

1. Design the database schema.
2. Implement the **RESTful API** and web server using Go, with a clean code hexagonal architecture and dependency injection pattern.
3. Implement role-based access control as a database trigger and HTTP middleware.
4. Implement logging and log rotation using Zap and Lumberjack.
5. Implement server best practices like graceful shutdown.

6. Implement the frontend with a server-side rendering interface using Templ and HTMX.
7. Implement LaTeX support and syntax highlight.
8. Implement a security model that solves the **OWASP Top 10** vulnerabilities.
9. Evaluate the system performance and utilization and compare it with another traditional architectural approaches.

1.6. Organization of the Project

- **Literature Review:** Provides a background, covering e-assessment systems, server-side rendering, database design patterns, architectural approaches, academic integrity mechanisms, and an analysis of related existing systems.
- **System Design and Implementation:** Describes the system design and implementation in detail, including the system architecture, technology selections, database schema, security mechanisms, and the implementation of core features.
- **Evaluation:** Presents an evaluation of the system, including architectural comparisons, performance measurements, and resource utilization analysis.
- **Conclusion and Future Work:** Concludes the thesis with a summary of contributions and directions for future work.
- **Appendices:** Provide source code snippets, the complete database schema, and user interface screenshots.



Chapter II

Literature Review



2.1. E-Assessment Systems

Electronic Assessment (E-Assessment) or **computer-based assessment**, is the use of *digital technologies* to measure and evaluate student learning outcome. The **e-assessment systems** have been on the rise for the past few years, caused by the rapid development and innovation in the *computer network* field, and the rise of the **internet**. Providing a replacement for the traditional *paper-based* exams.

2.1.1. The advantages of such a platform

1. The reduction or the removal of the administrative overhead.
2. The reduction of the printing costs and paper prices.
3. The preservation of data that is very hard or impossible to tamper with.
4. Flexible and Scalable Solution.
5. Grading time is not affected by the number of students.

2.2. Evolution of E-Assessment Systems

The first generation of computer-based exams was almost exclusively offline because of the lack of computer connectivity back then, to Make the examination software up and running; a school or an institution must install the software locally with the aid of a specialist in the software. That means the software was limited and costly.

After the rise of the **WWW (World Wide Web)**, an online examination system began to take shape with simple security measures like **RBAC (Role-Based Access Control)** [3].

Over the years, advances in computer-based online examinations have been remarkable and fast, shifting focus from the simple examination platform to platforms that are trying to solve problems like *Security, Scalability, and automation*.

In the **COVID-19** pandemic, the need for a *secure, convenient, automated, and performant* examination system was critical [2]. examination management systems increasingly adopt *cloud-native architectures, microservice*

decomposition, modern frontend and backend frameworks to address scalability, maintainability, and user experience challenges.

2.3. Static sites vs. Dynamic sites

2.3.1. Static sites

static site page or a flat page is a web page that is delivered to the end user without any modification on the page file content.

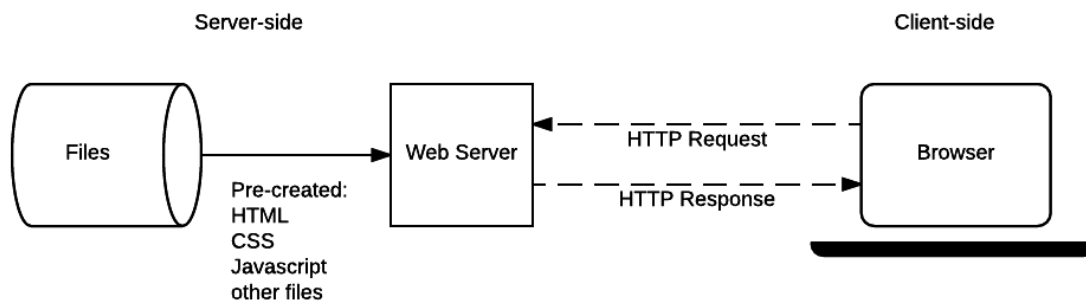


Figure 1: Static Sites Architecture

2.3.2. Dynamic sites

dynamic sites are web pages that are generated according to the user request, it aggregates data from the database and renders it.

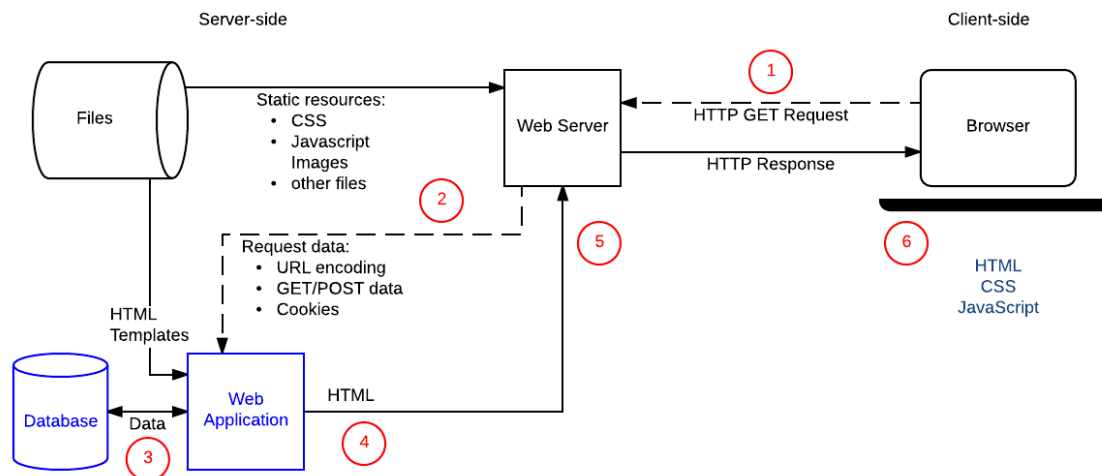


Figure 2: Dynamic Sites Architecture

The use of dynamic sites is more suitable for a data-driven application such as an electronic examination management system, it allows for more flexibility and interactivity.

2.4. Type-Safe Templating in Web Applications

The **template engine** is the base of building a dynamic *server-side rendered* web application. Template engines are crucial for separating *domain logic* from *the presentation*. The output of the template engine is usually **HTML** or **XML** [4].

2.4.1. Traditional Templating Engines

The traditional template engine that is used in programming languages like **Python** (**Jinja**), **PHP** (**Twig**), **Ruby** (**ERB**) uses the string interpolation technique to generate content at *runtime*. This technique is prone to security vulnerabilities like **XSS** (**cross-site scripting**), **SSTI** (**Server-side template injection**). The following code snippet is an example of a vulnerable **SSTI** template engine in **python**:

```
1 from flask import Flask, request,
   render_template_string
2
3 app = Flask(__name__)
4
5 @app.route('/greet')
6 def greet():
7     name = request.args.get('name', 'Guest')
8     template = f"<h1>Hello {name}</h1>"
9     return render_template_string(template)
```

 Python

Listing 1: An example of an SSTI vulnerable Python template engine source code.

It is quite easy to misuse the string interpolation technique. A caution is needed when developing with a type of template engine [4].

2.4.2. Type-Safe Templating Engines

On the contrary, the alternative to the traditional template engines is much stricter and more secure, by implication of *strong-typing* and stricter escaping rules, such as **Rust's (askama)** and **Go's (Templ)**, which are *compiled optimized template engines*.

```
1  package main
2
3  templ Hello(name string) {
4      <div>Hello, { name }</div>
5  }
6
7  templ Greeting(person Person) {
8      <div class="greeting">
9          @Hello(person.Name)
10     </div>
11 }
```

 Go

Listing 2: An example of a type-safe template engine in Go (Templ)

Templ is a *type-safe compiled* template engine that compiles the template files into **Go code**, then compiles it to a **binary**. This pattern gives a huge advantage; these advantages [5] include:

1. **Compile-time** type-safety.
2. Automatic **XSS** prevention.
3. Use of the **Go** *ecosystem and composition pattern*.
4. Performance and optimization.

2.5. Server-Side Rendering vs. Single-Page Applications

In modern web development, there are two main approaches to developing a web application: the first approach is the traditional *server-side rendering*, and the second approach, and the most dominant one in modern web development, is **SPA (Single Page Application)** using *JavaScript frameworks* like **Vue.js**, **React.js**, and **Angular.js**. While SPA provides a better

ecosystem and more interactivity due to the *dynamic compiled* updating without reloading the full page, it throws the weight of computation to the client to handle it. **SPA** are also more complex to implement with more *JavaScript bundles*; this complexity can also produce a *security risk and critical vulnerabilities* [6], [7].

While **SSR (Server-Side Rendering)** is a more traditional approach, it is still widely used in many applications, especially in *data-driven applications* like examination management systems. SSR frameworks like **Ruby on Rails**, and **ASP.NET**, focusing on *performance, security, and ease of implementation*. A *lightweight framework* like **HTMX** can be a middle ground between classic **SSR** and **SPA** with minimal JavaScript. **HTMX** can emulate the dynamic nature of **SPA** by utilizing **AJAX (Asynchronous JavaScript And XML)** to update parts of the page without reloading the whole page.

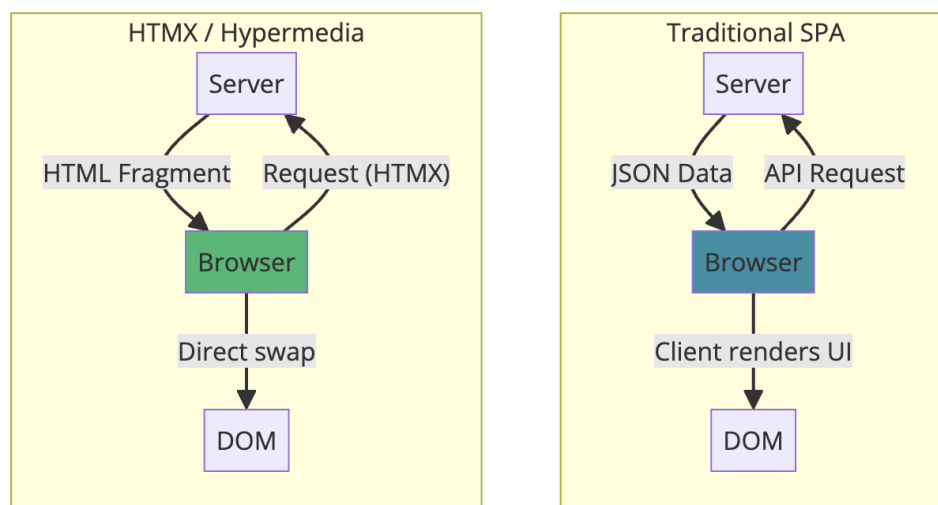


Figure 3: HTMX vs SPA Architecture

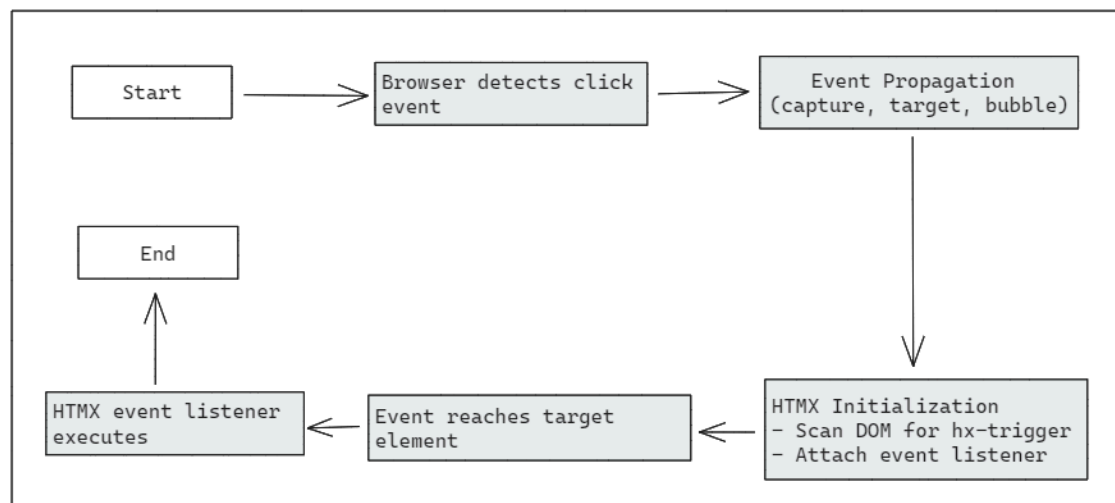


Figure 4: HTMX Flow Diagram

2.6. Database Design

To implement such a *performance and data-driven* application, a database with a *well-designed and optimized schema* must be designed and implemented. Such a design can be obtained by using the following principles:

1. Database normalization.
2. Database indexing.
3. Soft deletion and archiving.
4. Automatic data integrity with triggers and constraints.
5. Compiled queries and type-safety.
6. Database Transaction Management.

2.6.1. Database Normalization

Database normalization is the process of organizing the data in the database and reducing redundancy. Database normalization has 3 main normal forms:

1. **First Normal Form (1NF)**: The first normal form of a database implies that the data in the database be atomic where each field contains only one value.
2. **Second Normal Form (2NF)**: The second normal form of a database is when the database satisfies the condition of 1NF and all the non-key fields depend on the primary key completely.
3. **Third Normal Form (3NF)**: The third normal form of a database is when the database satisfies the conditions of 2NF and there are no transitive dependency among non-key fields.

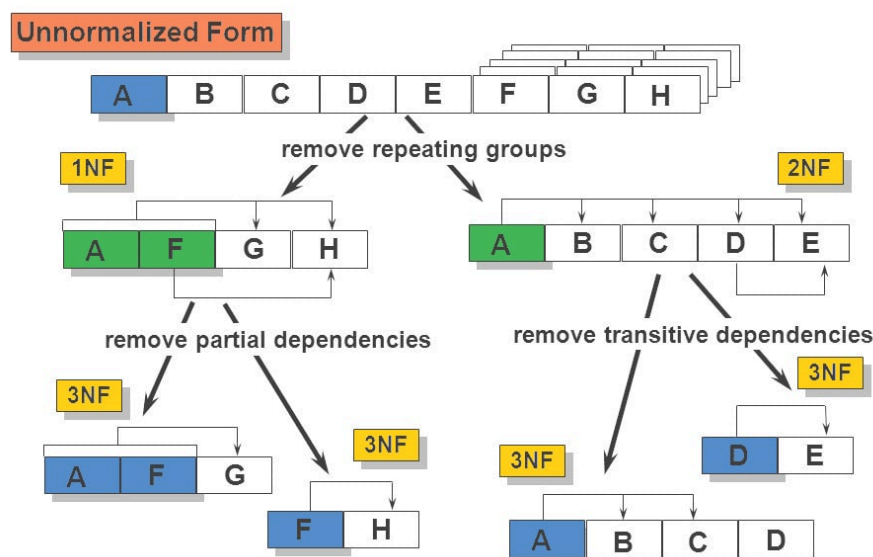


Figure 5: NF1, NF2, NF3 Forms

2.6.2. Database Indexing

Database indexing is an optimization data structure that improves the database performance. By creating an index for the most frequently used queries, a faster row find is possible [8].

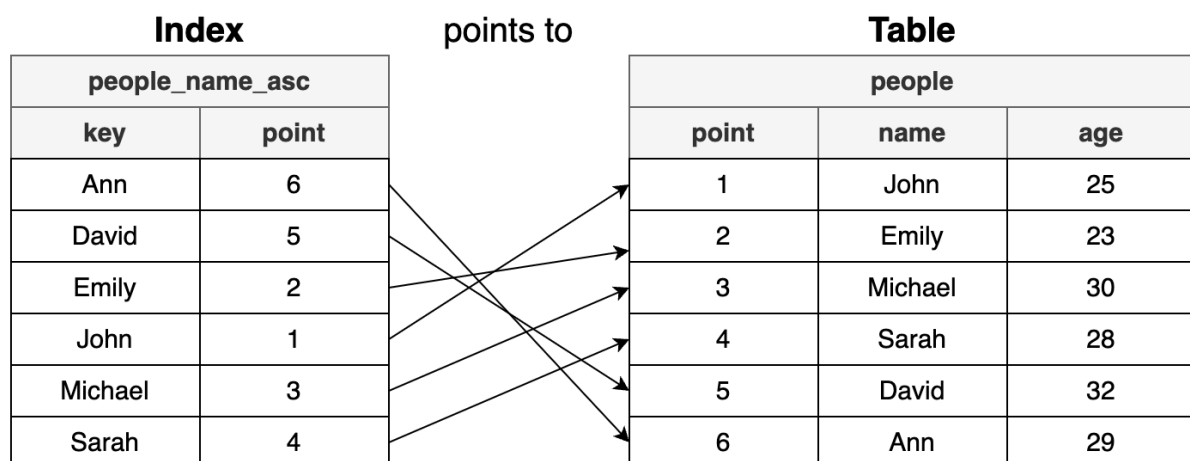


Figure 6: Indexing a database table with a Non-Clustered Index

2.6.3. Database Transaction Management

A database has a state of consistency. When a transaction is executed, the database state will temporarily change to an inconsistent state. A trans-

action manager is responsible for making sure that the database returns to a consistent state after the inconsistent period. By *rolling back* if the transaction fails, otherwise the transaction will be committed successfully.

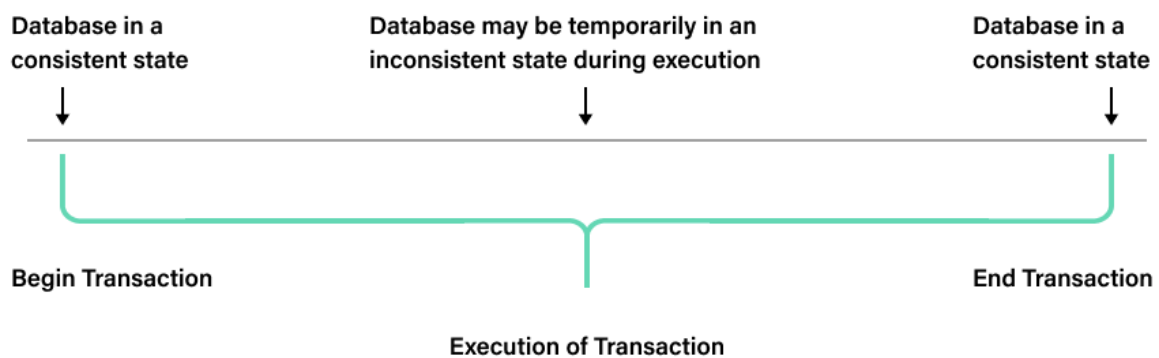


Figure 7: Transaction State Diagram

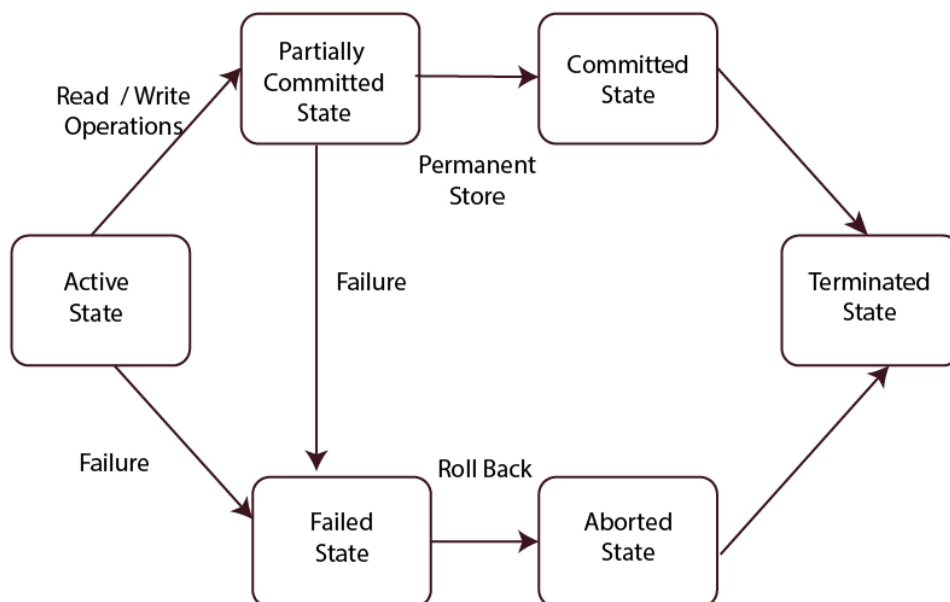


Figure 8: Transaction Manager Diagram

2.7. Compiled queries

In order to ensure the *type-safety* of a database query, an inline written query is not the best for *type-safety* and a maintainable code base. A better-suited solution is either to use an **ORM (Object Relational Mapping)** or a **compiled query builder**. The **ORM** approach is *more common* in modern web development. It gives the user a set of interfaces to implement and generate the desired **SQL query**. **ORMs** are a convenient, *easy-to-use* approach, but they *may lack control, flexibility*, and sometimes a lack of *optimization* on a certain query. On the other hand, a compiled query builder like **SQLC** is a more *flexible* and *optimized* approach; it gives the user more control over the generated **SQL query**, also supports the *repository pattern*, but it may require more effort to implement [9].

```
1 -- name: GetUserByID :one
2 SELECT * FROM user WHERE id = $1;
```

 SQL

The compiled code generated by SQLC would look like this:

```
1 const getUserByID = `-- name: GetUserByID :one
2 SELECT id, username, full_name, password, role FROM users
3 WHERE id = $1 AND deleted_at IS NULL
4 `
5
6 func (q *Queries) GetUserByID(ctx context.Context, id
7 uuid.UUID) (User, error) {
8     row := q.db.QueryRow(ctx, getUserByID, id)
9     var i User
10    err := row.Scan(
11        &i.ID,
12        &i.Username,
13        &i.FullName,
14        &i.Password,
15        &i.Role,
16    )
17    return i, err
18 }
```

 Go

Listing 3: Compiled queries with SQLC example

2.8. Architectural Patterns in Web Back-End Systems

2.8.1. MVC (Model-View-Controller)

The MVC architecture pattern is one of the most common architectural choices in web development. It separates the application into three distinct layers: the model layer, the Controller layer, and the view layer. Each layer is responsible for its own tasks.

1. **The Controller layer** is responsible for handling the user requests and responses; it acts as a mediator between the model and the view layers.
2. **The Model layer**: is responsible for handling the business logic and data manipulation.
3. **The View layer**: is responsible for rendering and presenting the data and the interface to the end user.

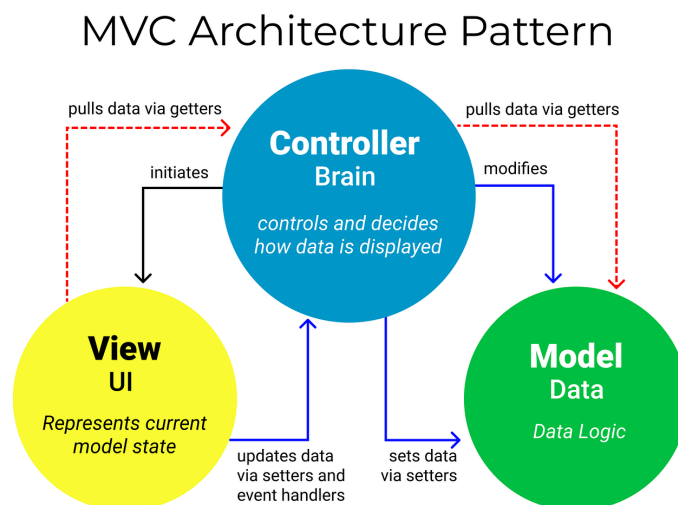


Figure 9: MVC Architecture Diagram

2.8.2. Hexagonal Architecture (Ports and Adapters)

Hexagonal architecture (Ports and Adapters) is a *software design pattern* first introduced by **Alistair Cockburn** in 2005 [10]. This architectural pattern is based on the idea of separating the application into different decoupled layers: *the core layer*, *the ports layer*, and *the adapters layer*. All

layers are wrapped in a *hexagonal shape* with *the core layer* in the center that has no dependencies on the other layers, and communicate with the outside world through *well-defined ports and adapters*.

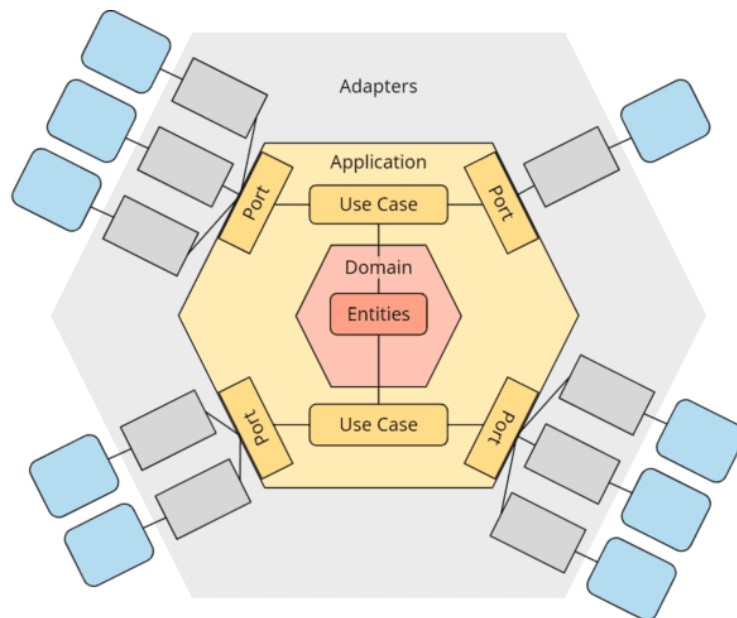


Figure 10: Hexagonal Architecture Diagram

The hexagonal architecture pattern is based on a principle called the **dependency inversion principle** [11], which is the last principle of the **SOLID** principles. That state :

High-level modules should not import anything from low-level modules. Both should depend on abstractions (e.g., interfaces).

Abstractions should not depend on details. Details (concrete implementations) should depend on abstractions.

2.9. Academic Integrity in Online Examinations

Online examinations are not like traditional *pen-and-paper* exams, which can be monitored somewhat easily. In a digital online examination *environment*, *academic integrity* measures must be implemented to ensure the *integrity* of the examination framework. Some of these measures are :

- Time limits and auto-submission.

- Question randomization.
- Question checksums.
- Audit logs and monitoring.

2.10. Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) is a security model that restricts access to resources based on users' roles. In the examination management system context, **RBAC** can be used to *restrict and separate* access to different functionalities based on the user role.

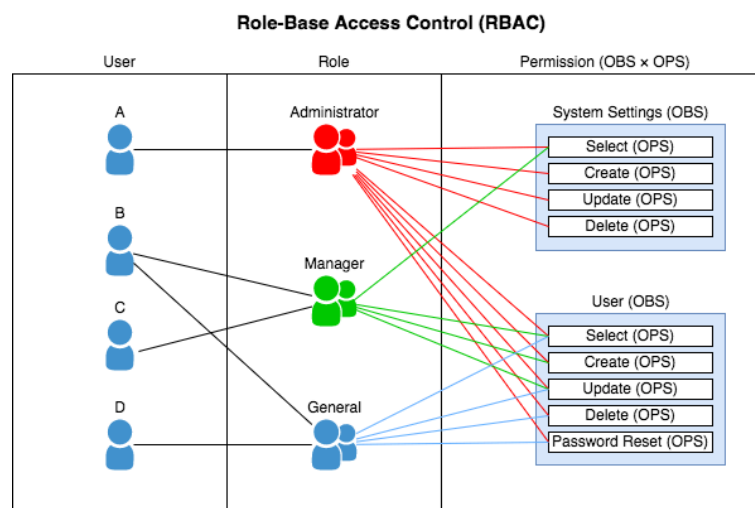


Figure 11: Role-Based Access Control Diagram

In this specific system, the **RBAC** is implemented into distinct layers. The *highest level layer* is implemented as an *HTTP middleware* and a **JWT-claims** using the **Go** programming language, which is responsible for checking the user role and allowing or denying access to the requested resource. The other layer is implemented as a *database constraints* using an *enum type* to restrict and validate roles `user_role_type`.

```
1 CREATE TYPE user_role_type AS
2 ENUM ('STUDENT', 'INSTRUCTOR', 'ADMIN');
```

SQL

2.11. User Interface

The user interface is designed using **Tailwind CSS** and **TemplUI** component library with **HTMX** for the interactivity. The UI is designed to be *user-friendly*.

The Use of **TemplUI** component library gives the UI a modern and unified look and feel. The library provides a high level and customizable components.

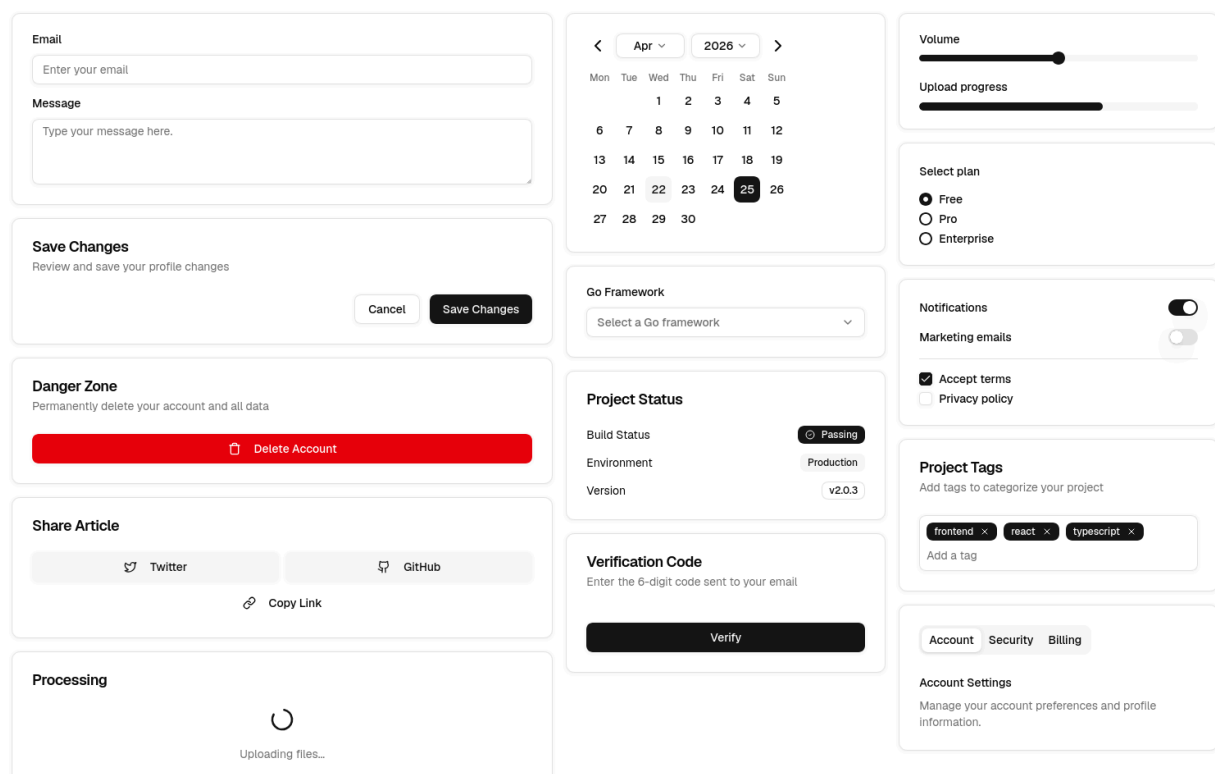


Figure 12: TemplUI Component Library

2.12. Related Work

2.12.1. Moodle

Moodle is a free and open-source learning management system (LMS) used for online education and training. It was developed by Martin Dougiamas and first released in 2002. It supports a various functionalities one of them is the quiz management system. The system is a web-based application based on a *PHP* programming language and a *MySQL* database.

But the system is very resource-intensive with a low performance and a outdated UI.

2.12.2. Google Forms and Microsoft Forms

Google Forms and **Microsoft Forms** are two of the most popular e-examination platforms. They both provide a simple and basic *MCQ format*. These platforms are lacking robust features like **RBAC**, **question checksums**, and **time-bound exams**. These platforms are storing all the response data in a cloud-based infrastructure over which the institution has no control.

2.12.3. Existing System (College of Computer Science)

The currently used system in the college of *computer science and computer engineering* is a **CSharp** application that connects with a *PHP* back-end using a *RESTful API*. The system uses an old stack of technologies, with *hardcoded* admin credentials, no *question checksums*, no *question randomization*, no *images*, no *mathematical notations*, and no *audit logs*. The UI is very outdated and not user-friendly. The system does not utilize any clean architectural patterns, and the code is not maintainable. The system also suffers from *segmentation faults* and memory leaks. The idea of this project is to solve the existing problems with a new *modern*, *secure*, *scalable*, and *performant* examination management system.



Chapter III

System Design and Implementation



3.1. Research Methodology

The development phase of this project was carried out using the **Agile Methodology** software development methodology. The system is a *web-based, data-driven application*. So, a **design-driven development** methodology was used to implement the system. At the analysis phase, the system requirements were gathered (**functional and non-functional requirements**) and analyzed. A **vertical slice** of the system was implemented first into layers :

1. **The Foundation Layer**: database schema, domain model, and repositories.
2. **The Application Layer**: application services and use cases.
3. **The Presentation Layer**: web handlers and templates.

Security was a major concept in the design phase of the system; **TLS**, **CSRF**, **JWT**, **HTTP Security Headers**, and **RBAC** were implemented to ensure the security of the system. The system was built using the **Hexagonal Architecture** pattern, which is a *domain-driven architecture* that focuses on the separation of concerns and *dependency decoupling*. The stack was used in the system to give the system a modern, fast, *well-documented, well-maintained, and easy-to-scale stack*. focusing on **performance** and **type-safety**.

3.2. System Design

3.2.1. System Architecture Overview

This system, as mentioned before, follows a Hexagonal architecture pattern, which separates code into separate domains:

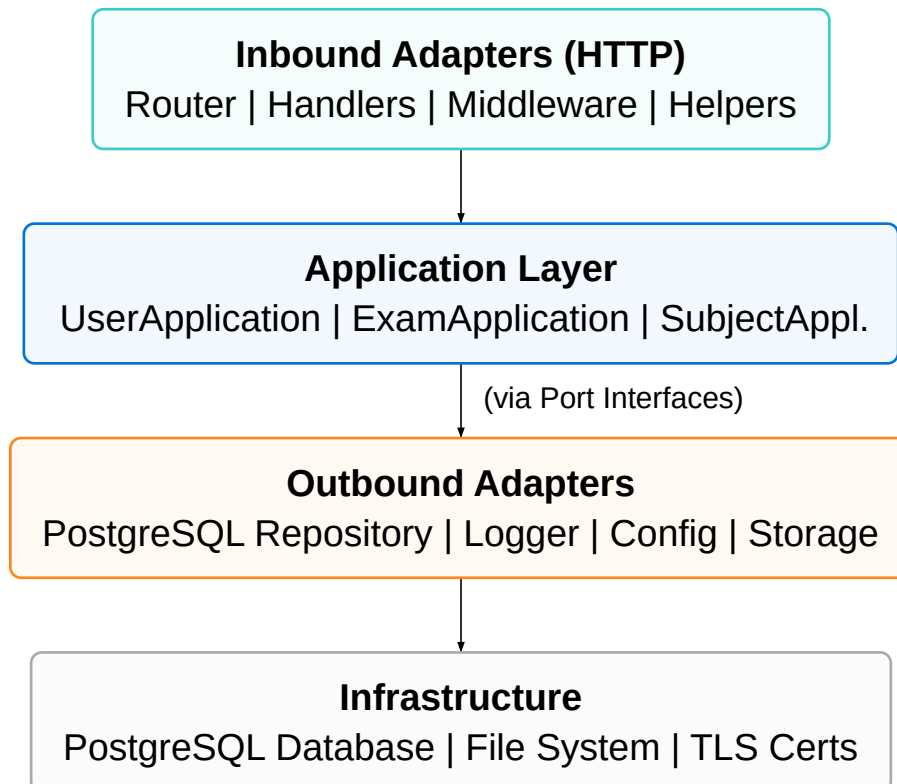


Figure 13: System Architecture Overview

- **Inbound Adapters**: Inbound adapters are often called **driving adapters**, because they drive the application. It is the only layer that interacts with the outside world. This system uses **HTTP** as the inbound adapter. This layer is responsible for handling the **HTTP** requests and responses.
- **Application Layer**: The application layer is responsible for the business logic of the system, and **transaction management**.

- **Outbound Adapters:** Outbound adapters are often called ***driven adapters***, because they drive the application. This layer is responsible for driving the database, file system, logs, and configuration.

3.2.2. Technology Selection and Justification

Component	Technology	Justification
Backend	Go 1.25.6	High-performance compiled language, modern ecosystem, type-safe, concurrency, ideal for enterprise-level applications
HTTP Framework	Gin	High-performance, middleware support, well documented and maintained
Database	PostgreSQL 18.3	Relational database, ACID compliance, scalability, database triggers, and advanced data types
Containerization	Docker	Industry standard for containerization
Frontend	HTMX	Lightweight interactivity framework with Ajax capabilities
Database driver	pgx/v5	Better Go driver for PostgreSQL than database/sql
Query Builder	SQLC	Generate type-safe Go code from a SQL query, and supports repository pattern
Templating Engine	Templ	Type-safe, compiles to Go code, secure from XSS attacks, and uses a component-based architecture
Frontend interactivity	HTMX	Lightweight interactivity framework with Ajax capabilities, no or minimal JavaScript, server-side rendering

CSS framework	Tailwind CSS	Utility-first CSS framework, well-documented and maintained
Database Migrations	Goose	Simple command-line tool for database migrations
Logging	Uber's Zap	High-performance, structured, and flexible configuration
Log rotation	Lumberjack	log file rotation, compression, and flexible configuration
Config	Viper	Flexible configuration management, supports multiple formats like yaml
Auth	JWT	Stateless authentication
Password Hashing	Bcrypt	Powerful hashing algorithm, that is the industry standard for password hashing
Session management	Gin-contrib/sessions	Session management for CSRF tokens
CSRF Protection	Gin-contrib/sessions	CSRF protection middleware for HTTP requests
TLS Encryption	net	TLS support for HTTP/2
PDF Generation	Maroto	Well documented and flexible PDF generator
CSV Generation	encoding/csv	CSV generation for data imports and exports
LaTeX Rendering	KaTeX	Fast and reliable LaTeX rendering

Code Highlighting	Highlight.js	High-performance, many languages and themes supported
-------------------	--------------	---

3.2.3. Database Design and Schema

The database is built using **PostgreSQL**, the main focus of the database is to be **normalized and efficient**. The database utilizes PostgreSQL plugins such as **pgcrypto** for **password hashing** and **uuid** generation, and **citext** for **case-insensitive text**. Also, the utilization of **database triggers** and **constraints** ensures the integrity of the database.

```
1 CREATE TABLE users (  
2   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3   username CITEXT NOT NULL UNIQUE,  
4   full_name VARCHAR(255) NOT NULL,  
5   branch VARCHAR(255) NOT NULL,  
6   password_hash TEXT NOT NULL,  
7   role user_role_type NOT NULL DEFAULT 'STUDENT',  
8   is_active BOOLEAN NOT NULL DEFAULT TRUE,  
9   last_login TIMESTAMPTZ,  
10  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
11  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
12  deleted_at TIMESTAMPTZ NULL,  
13  CHECK (char_length(username) >= 3),  
14  CHECK (username ~ '^[a-zA-Z0-9_]+$')  
15 );
```

Listing 4: User table schema

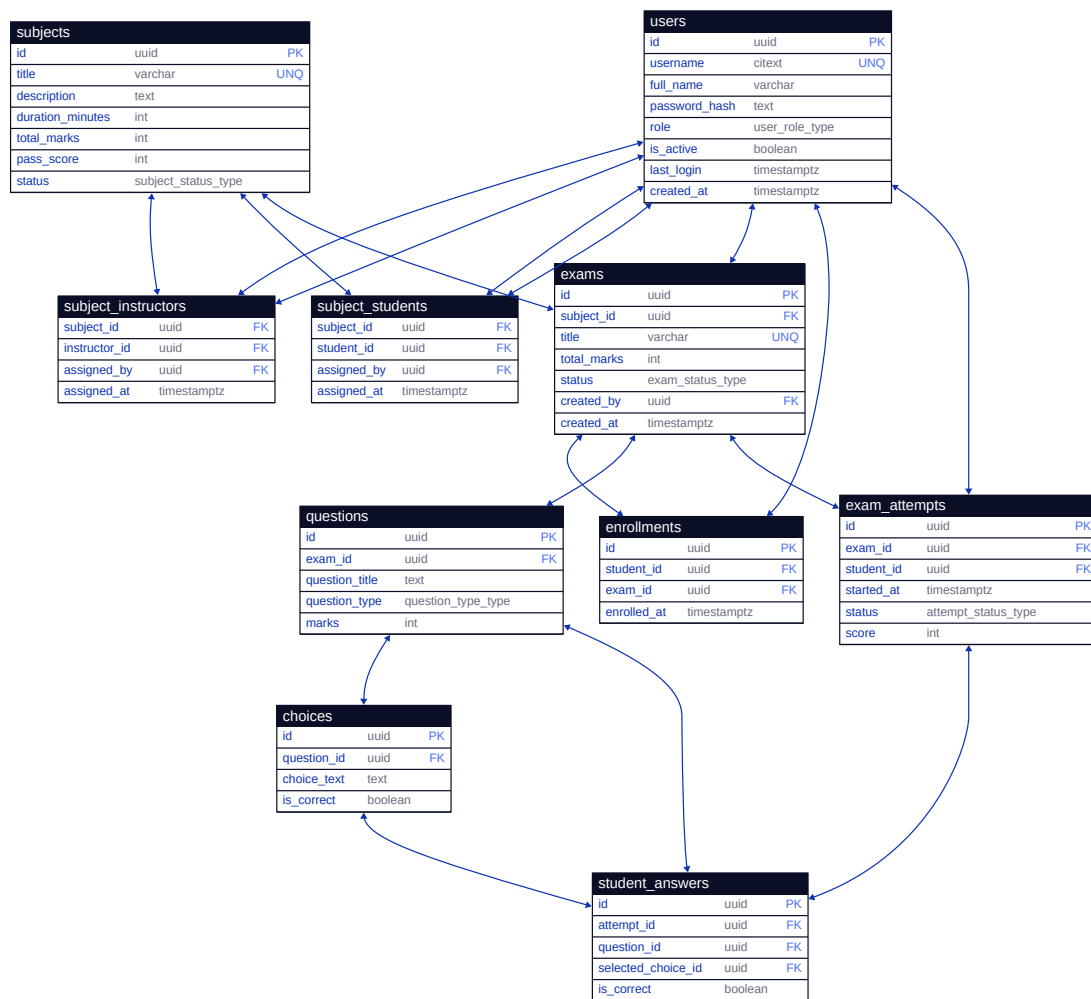


Figure 14: Database Entity-Relationship Schema

- **Users**: The users table is the central table of this system, which stores information about users that is used in authentication and authorization. The users can be a STUDENT, INSTRUCTOR, or ADMIN.
- **Collections**: The collections table is used to group exams together. The collections can be ACTIVE or INACTIVE.
- **Exams**: The exams table is responsible for storing information about exams. The exams can be in DRAFT, PUBLISHED, or CLOSED status. It will only be accessible to students if it is in PUBLISHED status and assigned to a collection that is ACTIVE.

- **Questions:** The questions table is used to store questions that are linked to a specific exam. That can be of TEXT, CODE, or IMAGE question type.
- **Choices:** The choices table is used to store the choices for the questions.
- **Collection_instructors:** The collection_instructors table is used to store a relationship between instructors and collections, which means the administrator can assign a collection to an instructor.
- **Collection_students:** The collection_students table is used to store a relationship between collections and students, similar to the collection_instructors table. The administrator can assign a collection to a student.
- **Student_answers:** The student_answers table is used to store the answers of students for each question at each exam attempt.
- **Exam_attempts:** The exam_attempts table is used to store information about exam attempts of students. It has three statuses: IN_PROGRESS, SUBMITTED, and CANCELLED.

3.2.4. User Roles and Access Control Mechanism

The system has three user roles: STUDENT, INSTRUCTOR, and ADMIN. Each role has its own set of privileges and user interface. These privileges can be listed as follows:

Role	Interface	Privileges
Student	Student Dashboard	• View assigned exams • Attempt exams • View results • Export results as PDF
Instructor	Instructor Dashboard	• Create/edit/delete exams • add/edit/delete questions • assign/unassign collections to students
Admin	Admin Dashboard	• Create/edit/delete collections • add/delete/restore users • assign/unassign collections to instructors and students

		• Create/edit/delete exams • add/edit/delete questions
--	--	--

The mechanism by which the system checks if the user is authorized to access a specific resource or not is to use *HTTP middlewares*. In this system, the authorization is done by chaining two middleware functions:

1. `AuthenticationMiddleware()`: This middleware is responsible for checking the JWT token of the request and storing the claims in the gin context.
2. `RoleAuthMiddleware(allowedRoles...)`: This middleware takes the user role from the stored claims in the gin context and checks if the user is authorized to access the requested resource and if the user is active or not.`

```
func (m *AuthMiddleware)
1 RoleAuthMiddleware(allowedRoles ...domain.UserRole)
  gin.HandlerFunc {
2   roleSet := make(map[domain.UserRole]struct{},
    len(allowedRoles))
3   for _, r := range allowedRoles {
4     roleSet[r] = struct{}{}
5   }
6
7   return func(c *gin.Context) {
8     raw, exists := c.Get("claims")
9     if !exists {
10      c.Abort()
11      return
12    }
13    if !claims.IsActive {
14      c.Abort()
15      return
16    }
17    userRole := domain.UserRole(claims.Role)
18    if _, allowed := roleSet[userRole]; !allowed {
19      c.Abort()
20      return
21    }
22    c.Next()
23  }
24 }
```



Listing 5: Role-based authentication middleware

3.2.4.1 System Flowchart

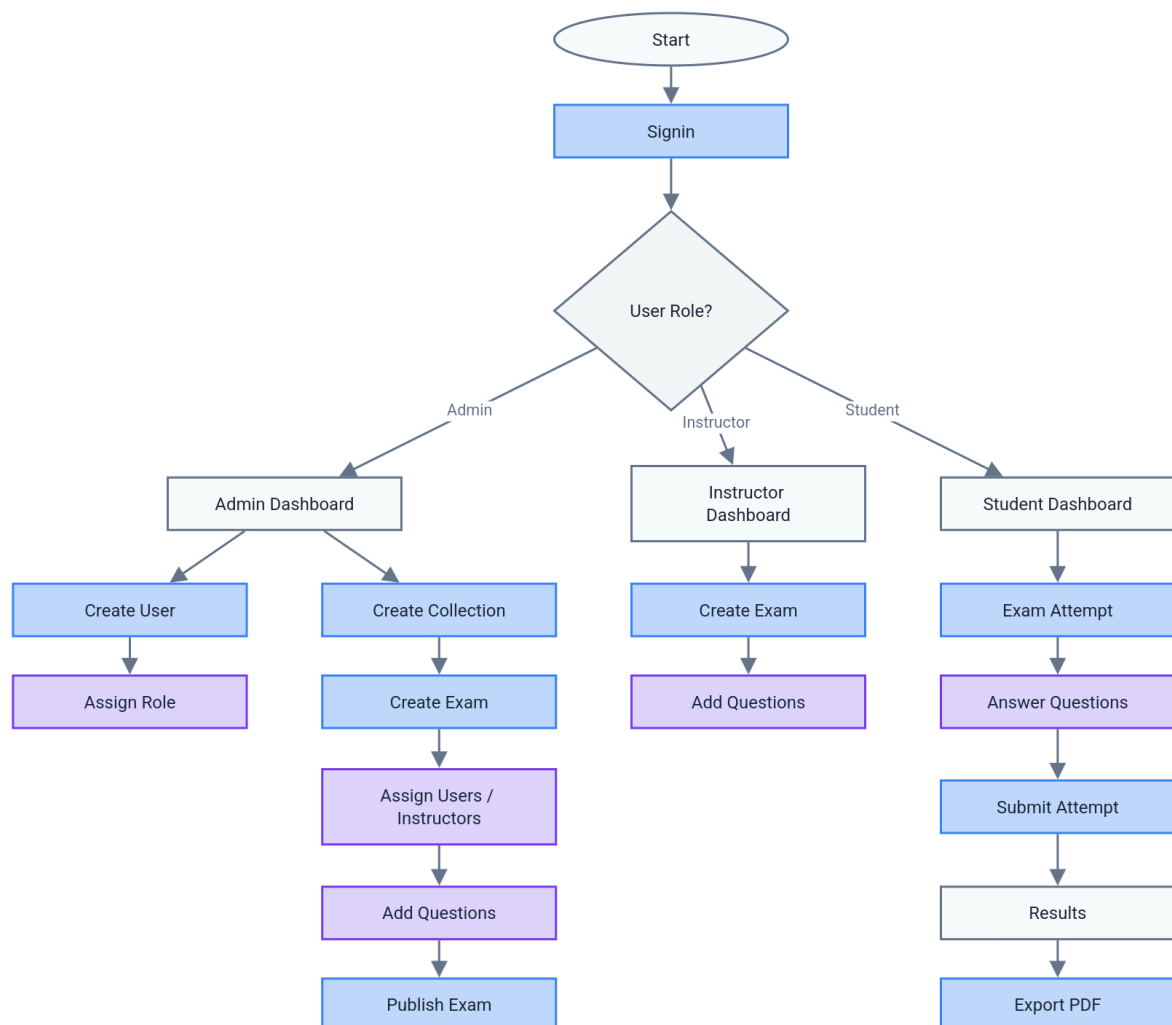


Figure 15: System Workflow and User Interactions

3.2.5. System Configuration

To make the system modifiable and configurable, a config file system is used. The configuration is stored in a **YAML** file located at `/config/config.yaml`. The configuration has default values for all of the parameters if none of the parameters are provided to prevent any errors during the startup of the system.

```
1  http_server:
2    address: "localhost"
3    port: "8443"
4    tls_cert_dir: "certs"
5    cert_file: "certs/cert.pem"
6    key_file: "certs/key.pem"
7
8  database:
9    host: "localhost"
10   port: 5432
11   username: "testuser"
12   password: "123456"
13   database_name: "testdb"
14   ssl_mode: "disable"
15   max_connections: 25
16   min_connections: 5
17   max_conn_lifetime: 1h
18   max_conn_idle_time: 30m
19
20  // rest of the config file
```

 **YAML**

Listing 6: System Configuration File

3.2.6. Deployment Strategy

To deploy the system with a *modern, efficient, and scalable solution*, a containerization strategy is a must. The use of **docker** allows us to package and deploy the system in a dependency-free, isolated environment.

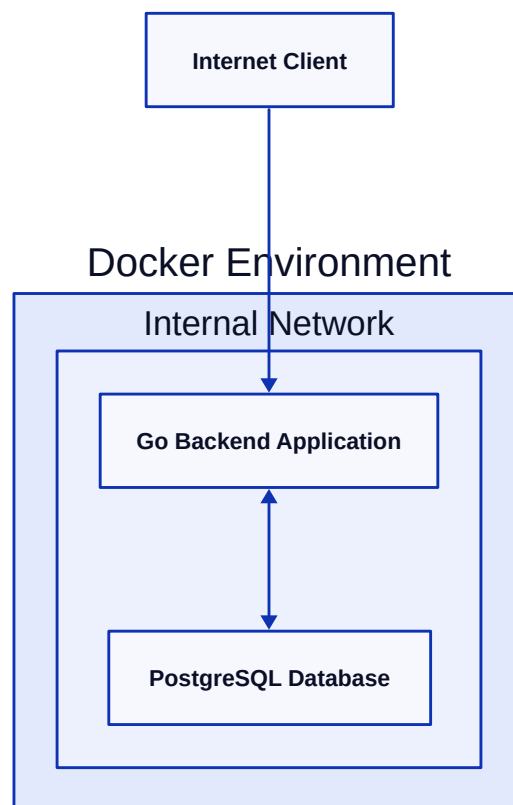


Figure 16: Docker Container Deployment Architecture Workflow

3.3. System Implementation

To provide a comprehensive understanding of the implementation of the system, we can trace a single event through the system, such as getting

a list of all users in the system by utilizing a method called `GetUsers()`, for example.

3.3.1. The Domain Layer

In the domain layer located at `/internal/domain`, we define the entities and business logic of the system with no dependencies or knowledge of the outside world.

```
1 type User struct {
2     ID          string    `json:"id"`
3     Username    string    `json:"username"`
4     FullName    string    `json:"full_name"`
5     Role        UserRole   `json:"role"`
6     IsActive    bool      `json:"is_active"`
7     CreatedAt   time.Time `json:"created_at"`
8     UpdatedAt   time.Time `json:"updated_at"`
9 }
```

Listing 7: Domain Layer Entity

3.3.2. The Port Layer

The Port layer `/internal/ports` works as an interface between the domain layer and the outside world. That way the domain layer is not dependent or aware of the outside world.

```
1 type UserRepository interface {
2     GetUsers() ([]User, error)
3 }
```

Listing 8: Port Layer Interface

3.3.3. The Infrastructure Layer (Outbound Adapter)

This layer `/internal/adapters/outbound` is the lowest level of implementation. This layer contains the implementation of the ports defined in the port interface. In this system, **SQLC** is already generating the repositories for us, but the system is now dependent on **SQLC** to solve this dependency issue. A better solution is to map the generated sqlc structs to the domain structs in another repository layer.

```
1 func (r *UserRepository) ListAll(ctx context.Context,  
  arg ports.ListAllUsersParams) ([]domain.User, error) {  
2   queries := r.queries  
3   if tx := database.ExtractTx(ctx); tx != nil {  
4     queries = queries.WithTx(tx)  
5   }  
6  
7   users, err := queries.ListAllUsers(ctx,  
    sqlc.ListAllUsersParams{  
8     Limit:  arg.Limit,  
9     Offset: arg.Offset,  
10  })  
11  if err != nil {  
12    return nil, err  
13  }  
14  
15  var domainUsers []domain.User  
16  for _, user := range users {  
17    domainUsers = append(domainUsers, domain.User{  
18      ID:      user.ID,  
19      Username: user.Username,  
20      FullName: user.FullName,  
21      Role:    domain.UserRole(user.Role),  
22      IsActive: user.IsActive,  
23      LastLogin: toTimePtr(user.LastLogin),  
24      CreatedAt: user.CreatedAt.Time,  
25      UpdatedAt: user.UpdatedAt.Time,  
26      DeletedAt: toTimePtr(user.DeletedAt),  
27    })  
28  }  
29  
30  return domainUsers, nil  
31 }
```



This repository method satisfies the port interface.

3.3.4. The Application Layer

The application layer `/internal/adapters/inbound` is the wrapper around the port interface and the highest level of implementation of the system. The application layer doesn't have any knowledge of what storage or what database is being used.

```
1 func (app *Application) ListAllUsers(ctx context.Context,
2   arg ports.ListAllUsersParams) ([]domain.User, error) {
3   users, err := app.userRepo.ListAll(ctx, arg)
4   if err != nil {
5       return nil, err
6   }
7   return users, nil
8 }
```

 Go

3.3.5. The Presentation Layer (Inbound Adapter)

The presentation layer `/internal/adapters/inbound` is the layer that directly interacts with the outside world. It deals with the incoming and outgoing data of the system.

```
1 func (h *UserHandler) ListAllUsers() gin.HandlerFunc {
2   return func(ctx *gin.Context) {
3       users, err := h.App.ListAllUsers(ctx,
4         ports.ListAllUsersParams{Limit: 100, Offset: 0})
5       if err != nil {
6           helpers.Toast(ctx, "List All Users Failed", "Failed to list
7             all users: " + err.Error(), toast.VariantError)
8       }
9       return
10  }
11 }
```

 Go

3.4. User Interface

This system is built using a component-based architecture with **Templ** and **HTMX**. These components can be triggered by the handlers in the *presentation layer*. For example, a ***Toast Notification*** is a component that can be triggered from the handler using **HTMX AJAX** Headers HX-Trigger and HX-Reswap.

```
1 func (h *UserHandler) ListAllUsers() gin.HandlerFunc {
2     return func(ctx *gin.Context) {
3         users, err := h.App.ListAllUsers(ctx,
4             ports.ListAllUsersParams{Limit: 100, Offset: 0})
5         if err != nil {
6             helpers.Toast(ctx, "List All Users Failed", "Failed to list
7                 all users: " + err.Error(), toast.VariantError)
8         }
9         return
10    }
11    render.Render(ctx, components.UserTableRows(users, false))
12 }
```



Listing 9: Toast Notification In Presentation Layer

```
1 func Toast(ctx *gin.Context, title string, description  
string, variant toast.Variant) {  
2 ctx.Writer.Write([]byte(`<div hx-swap-oob="beforeend:#toast-  
container">`))  
3  
4 toast.Toast(toast.Props{  
5 Title:      title,  
6 Description: description,  
7 Variant:    variant,  
8 Duration:   4000,  
9 ShowIndicator: true,  
10 Dismissible: true,  
11 Icon:       true,  
12 }).Render(ctx.Request.Context(), ctx.Writer)  
13  
14 ctx.Writer.Write([]byte(`</div>`))  
15 }
```



Listing 10: Toast Notification Implementation

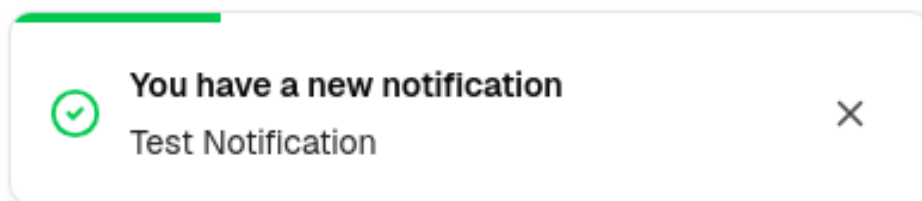


Figure 17: Toast Notification After Triggering

3.5. Security Mechanisms

3.5.1. OWASP Top 10 Security Considerations

OWASP Top 10 is a list of the most important and critical security risks for web applications [12]. The system is designed with these risks mitigation in mind.

OWASP Top 10 Risk	Mitigation Strategy
-------------------	---------------------

A01:2025 Broken Access Control	Role-Based Access Control (RBAC) middleware verifying allowed roles from JWT claims.
A02:2025 Security Misconfiguration	Strict TLS encryption enforcement, HTTP security headers (X-Content-Type-Options, Frame-Options), and configuration via Viper.
A03:2025 Software Supply Chain Failures	Use of Go's built-in module management (<code>go.mod</code> , <code>go.sum</code>) to strictly lock and verify dependency checksums.
A04:2025 Cryptographic Failures	Bcrypt algorithm for resilient password hashing; robust JWT token signing; enforcing <code>HttpOnly</code> and <code>Secure</code> cookie flags over HTTPS.
A05:2025 Injection	SQLC type-safe prepared statements natively eliminating SQL injection; <code>Templ</code> strictly escaping all dynamic HTML inputs against XSS.
A06:2025 Insecure Design	Hexagonal architecture isolating business logic from external frameworks, fortified with strict layer boundary separation.
A07:2025 Authentication Failures	Stateless JWT tokens with rigorous expiration deadlines; automated CSRF protection using <code>gin-csrf</code> rotating tokens.
A08:2025 Software or Data Integrity Failures	Data integrity enforced via database <code>CHECK</code> constraints, unique keys, structural triggers, and soft-deletion tracking.
A09:2025 Security Logging and Alerting Failures	Persistent, leveled, and structured JSON audit logging utilizing Uber's

	Zap logger matched with Lumberjack log rotation.
A10:2025 Mishandling of Exceptional Conditions	Centralized robust error-wrapping across all Adapters to prevent raw DB driver stack traces from leaking to clients.

3.5.2. SQL Injection Prevention Techniques

SQL injection is a critical vulnerability that exploits the escape characters in the database queries [13]. This system uses type-safe prepared statements generated by **SQLC** to prevent SQL injection attacks. The **SQLC** takes a .sql file and generates a type-safe Go code with a repository layer that used to interact with the database.

3.5.3. TLS Encryption

HTTP traffic goes through **HTTPS** with the help of **TLS** certificates located at cert.pem and key.pem in certs/. The implementation that **Go** standard library uses natively (**net/tls**), which is compatible with TLS 1.2 and TLS 1.3. The **HTTP** response header `Strict-Transport-Security max-age=31536000; includeSubDomains` is added to ensure that browsers will exclusively connect to the domain and its subdomains using **HTTPS** for a year from the first visit (**HSTS**).

The `Referrer-Policy: strict-origin-when-cross-origin` header limits the referrer information sent in cross-origin requests, preventing the leakage of sensitive URL parameters to external sites.

The system uses a self-signed certificate for local deployment and the ability to use a signed certificate from the configuration **YAML** file.

```
1 privateKey, err := rsa.GenerateKey(rand.Reader, 2048)
2 if err != nil {
3     return "", "", fmt.Errorf("failed to generate private key: %w", err)
4 }
5
6 template := x509.Certificate{
```



```
7  SerialNumber: big.NewInt(1),
8  Subject: pkix.Name{
9  Organization: []string{"University of Technology"},
10 Country:      []string{"IQ"},
11 Locality:     []string{"baghdad"},
12 },
13 NotBefore:    time.Now(),
14 NotAfter:     time.Now().Add(365 * 24 * time.Hour),
15 KeyUsage:     x509.KeyUsageKeyEncipherment |
x509.KeyUsageDigitalSignature,
16 ExtKeyUsage:
[]x509.ExtKeyUsage{x509.ExtKeyUsageServerAuth},
17 BasicConstraintsValid: true,
18 }
```

3.5.4. System Logging and Monitoring

The system utilizes structured logging with Uber's Zap library to log all the system events. The logs are stored in a file with a rotation mechanism to prevent the log file from growing indefinitely using lumberjack package, which is a compression and rotation mechanism for log files. **Zap** is a *near-zero allocation, fast, structured* logging library for **Go**.

```
1  {
2  "level":"INFO",
3  "timestamp":"2026-03-08T23:31:25.732+0300",
4  "caller":"server/server.go:74",
5  "message":"INTERNAL_INFO",
6  "info_msg":"Using an existing TLS certificate..."
7  }
```

 JSON

Listing 11: System Log Example

The use of **JSON** format for logging makes it easier to parse and analyze the logs using tools like **ELK Stack** or **Splunk**.

The system also has a separate log for all the routes and the **HTTP** requests.

```
1 [GIN] 2026/04/14 - 10:02:56 | 200 | 4.992395ms | 127.0.0.1 | GET "/admin/dashboard/users"
2 [GIN] 2026/04/14 - 10:02:59 | 200 | 3.799645ms | 127.0.0.1 | GET "/admin/dashboard/"
3 [GIN] 2026/04/14 - 10:03:01 | 200 | 2.919365ms | 127.0.0.1 | GET "/admin/dashboard/subjects"
```

Listing 12: System Route Log Example

3.5.5. Password Security

The system uses the **bcrypt** algorithm for password hashing. **bcrypt** is a cryptographic password hashing function based on the *Blowfish cipher*, designed by **Niels Provos** and **David Mazières** to protect against brute-force attacks. It is considered the industry standard for password hashing. **bcrypt** can be configured with a *cost factor*, the higher the cost factor, the more secure the password is, but the longer it takes to hash the password.

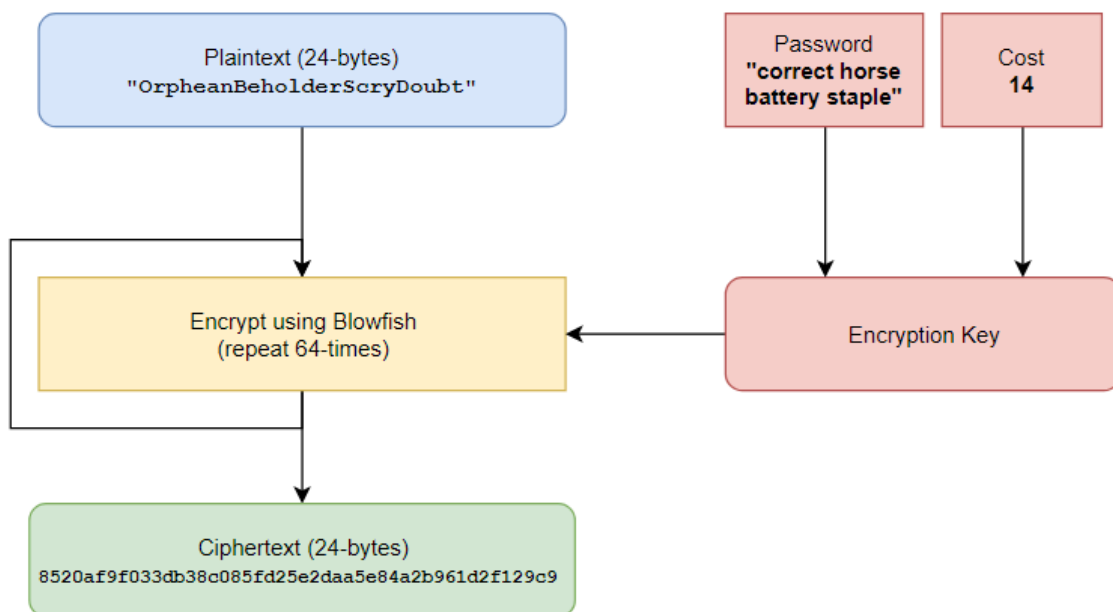


Figure 18: Bcrypt Password Hashing Algorithm

The system also validates the password strength using a set of rules that are defined in the system. The rules are as follows:

- Password must be at least 8 characters long.
- Password must contain at least one number.
- Password must contain at least one uppercase letter.

```
1 func ValidatePasswordCriteria(password string) error {
2     if password == "" {
3         return fmt.Errorf("password is required")
4     }
5     if len(password) < 8 {
6         return fmt.Errorf("password must be at least 8 characters
7         long")
8     }
9     var hasNum, hasUpper bool
10    for _, char := range password {
11        if unicode.IsNumber(char) {
12            hasNum = true
13        }
14        if unicode.IsUpper(char) {
15            hasUpper = true
16        }
17    }
18    if !hasNum || !hasUpper {
19        return fmt.Errorf("password must contain at least one number
20        and one uppercase letter")
21    }
22    return nil
23 }
```

Listing 13: Password Validation Function



Chapter IV

Results and Evaluation



4.1. System Architecture Evaluation

4.1.1. Comparison: Monolithic MVC vs. Hexagonal Architecture

This section evaluates this system's architecture (**Hexagonal Architecture**) against the monolithic **MVC** architecture like the one used in **Moodle**.

Feature	MVC	Hexagonal Architecture
Coupling	The model is tightly coupled to the view and controller.	The core domain is decoupled from the dependencies of the other system layers.
Testability	Hard to test due to the tight coupling between layers. If a function that requires a database connection is tested, it must have a database connection.	Easy to test due to the separation of concerns and layers. A mock database connection can be used to test the same function without a database connection.
Replaceability	Changing the database or web framework requires a system-wide rewrite.	Changing the database or web framework is easy and requires rewriting only the adapters without touching the core domain of the system.
Complexity	Lower initial complexity	Higher initial complexity
Framework lock-in	The framework is tightly coupled to the system, making it difficult to switch to a different framework.	The framework is decoupled from the core domain of the system, and only located in the adapters layer

The hexagonal architecture approach seems at first glance to be more complex and more verbose than the traditional **MVC**. But, it provides a better architectural solution when the codebase grows, and the system becomes more *complex*.

4.2. Performance Evaluation

The system performance was evaluated using:

- **Grafana K6:** A load testing tool for **APIs**, **microservices**, and **websites**.

The hardware used for the evaluation is:

- CPU: AMD Ryzen 7 4800H (16 Threads) @ 4.30GHz
- RAM: 16GB
- GPU: NVIDIA RTX 3060
- OS: Arch Linux 64-bit (Kernel 7.0.0-1-cachyos)
- Docker Engine: 29.4.0

The **k6** script is as follows:

```
1  import http from 'k6/http';
2  import { check, sleep } from 'k6';
3
4  export const options = {
5    vus: 100,
6    duration: '30s',
7  };
8
9  export default function () {
10   const res = http.get('https://localhost:8443/health', {
11     tlsInsecureSkipVerify: true,
12   });
13   check(res, {
14     'status is 200': (r) => r.status === 200,
15   });
16   sleep(1);
17 }
```

JS JavaScript

4.2.1. Response Time

A different request endpoint was tested and aggregated into a **CSV** file, then visualized.

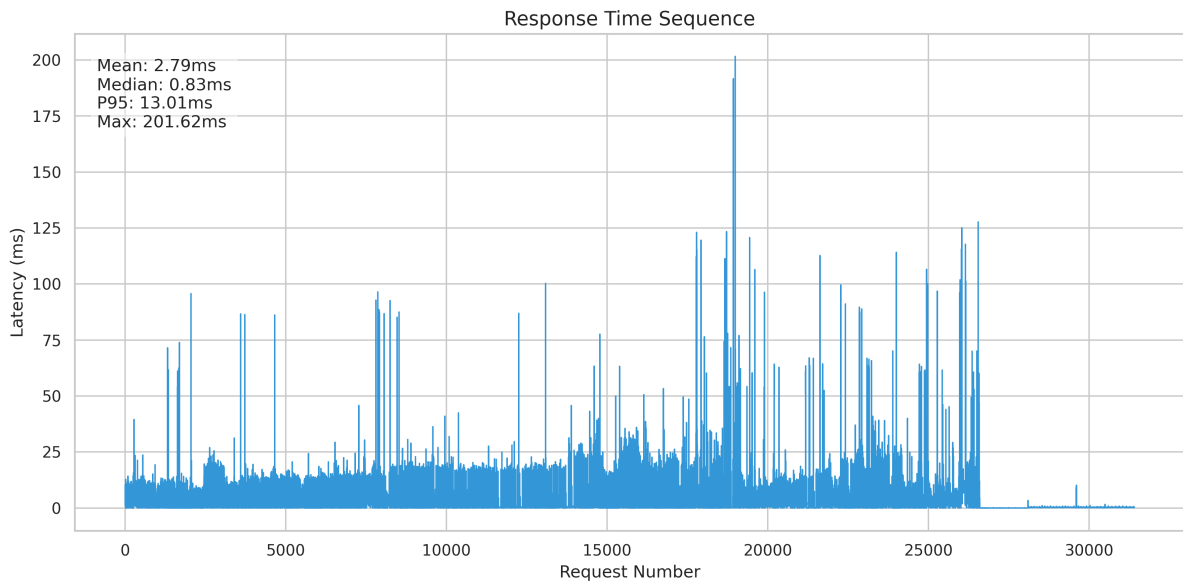


Figure 19: Response time of the system under load using **k6** and **Matplotlib**

4.2.2. Memory Usage

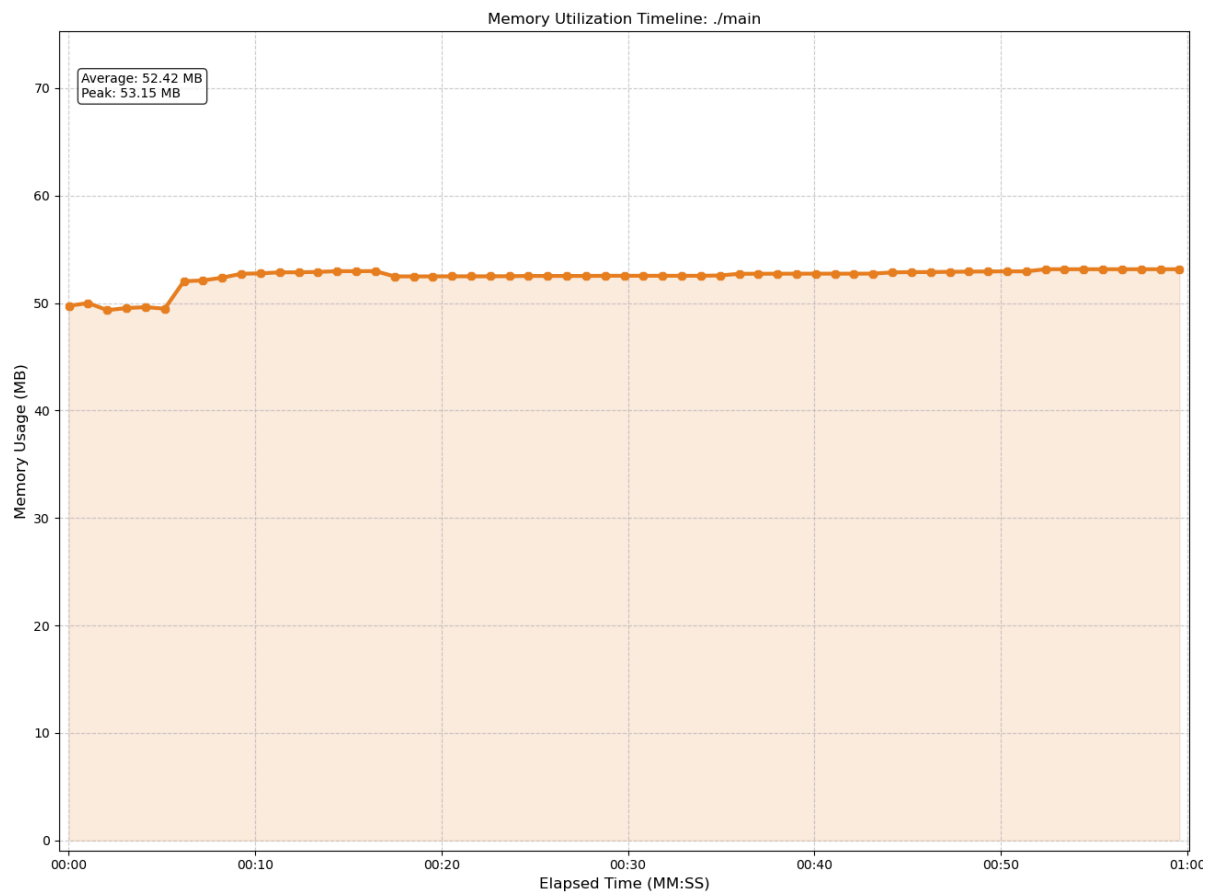


Figure 20: Memory usage of the system under load using **k6**

4.3. Comparison with Moodle, and MCQUOT

Metric	Moodle	MCQUOT	This System
Response Time (Average)	700ms	660ms	3ms
Memory Usage (Idle)	97MB	83MB	35MB
Memory Usage (Peak)	1GB to every 15 users	1GB to every 20 users	52MB to every 100 users
Startup time	1-2s	1-2s	<100 ms
Language	PHP	PHP	Go
Concurrency model	Multi-process	Multi-process	Goroutines (lightweight; M:N scheduling)
Type safety	Dynamic typing	Dynamic typing	Strong static typing
Binary size	N/A	N/A	60MB (self-contained binary)
Source code size	434.8 MB	422.9 MB	58.4 MB

4.4. Solved Problems of the Previous MCQUOT System

The previous system was suffering from a lot of *architectural*, *performance*, *security*, and *functional* problems. Some of these functional problems are:

4.4.1. Database question duplication

The previous system had no mechanism to prevent *duplication* of questions in a single exam context. So the same question could appear multiple times in the same exam.

To prevent this, we added a constraint that computes the MD5 hash of the question text and the exam `uuid` to ensure that there is no duplication of questions in the same exam.

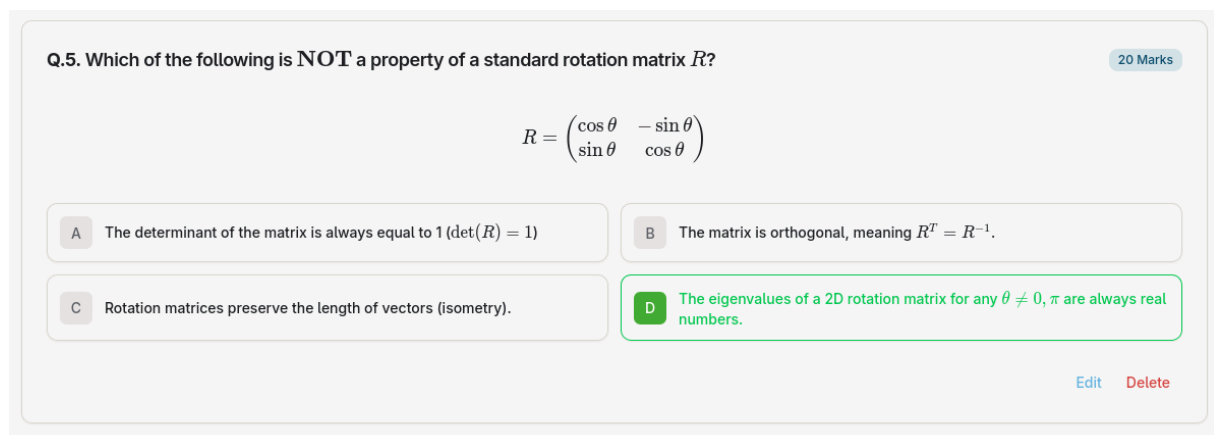
4.4.2. Database question randomization

The previous system had no mechanism to *randomize* the questions. To *randomize* the questions in a single exam attempt, we added a hash of the exam's `uuid` with the exam_attempt's `uuid` to generate a random seed for the random number generator to order the questions on it.

4.4.3. Question limitation

The previous system had only one type of question, and it was simply the text question type. This system supports multiple types of questions, such as:

4.4.3.1 Text questions



Q.5. Which of the following is **NOT** a property of a standard rotation matrix R ? 20 Marks

$$R = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

A The determinant of the matrix is always equal to 1 ($\det(R) = 1$)

B The matrix is orthogonal, meaning $R^T = R^{-1}$.

C Rotation matrices preserve the length of vectors (isometry).

D The eigenvalues of a 2D rotation matrix for any $\theta \neq 0, \pi$ are always real numbers.

[Edit](#) [Delete](#)

Figure 21: Text question type

4.4.3.2 Code highlighted questions

Q.1. What the following code is using ? 1 Marks

```
fn main() {  
    use std::collections::HashMap;  
  
    let mut scores = HashMap::new();  
    scores.insert(String::from("Blue"), 10);  
  
    scores.entry(String::from("Yellow")).or_insert(50);  
    scores.entry(String::from("Blue")).or_insert(50);  
  
    println!("{:?}", scores);  
}
```

☐ A Structs ☒ B Hashmaps ☐ C Vec3 ☐ D Classes

[Edit](#) [Delete](#)

Figure 22: Code highlighted question type

4.4.3.3 Image questions

Q.3. Which of the following sequences of input values represents a valid path that starts at *Start* and ends at an q^F state? 1 Marks

```

graph LR
    start((start)) --> q0((q0))
    q0 -- a --> q10((q10))
    q0 -- b --> q01((q01))
    q10 -- a --> q20((q20))
    q10 -- b --> q11((q11))
    q20 -- a --> q20
    q20 -- b --> q21((q21))
    q21 -- a --> q21
    q21 -- b --> qF(((qF)))
    q01 -- a --> q11
    q01 -- b --> q02((q02))
    q11 -- a --> q21
    q11 -- b --> q12((q12))
    q02 -- a --> q12
    q02 -- b --> q02
    q12 -- a --> qF
    q12 -- b --> q12
    qF -- "a, b" --> qF
  
```

Options:

- A $a \rightarrow b \rightarrow a \rightarrow a$
- B $b \rightarrow b \rightarrow a \rightarrow a$
- C $a \rightarrow a \rightarrow b \rightarrow b$**
- D $b \rightarrow a \rightarrow b \rightarrow a$

[Edit](#) [Delete](#)

Figure 23: Image question type

4.4.4. Manual Data Entry

The previous system had no mechanism to automate the data entry process. This system is solving this problem by implementing a **CSV import/export** feature to add questions, exams, users, and subjects to the system.



Chapter V

Conclusion and Future Work



5.1. Conclusion

This thesis presents a design and implementation of a web-based examination management application that is fast, secure, and easy to use. The system is built with modern technologies and clean architectural patterns to provide a reliable, fast, and scalable solution for managing examinations. The system is solving the problems, such as:

- **Automated grading and scoring:** The system grading process is automatic, with the ability to download the results as a PDF file.
- **Data integrity:** The integrity of the system is ensured by database constraints, the application layer, and HTTP middleware.
- **Security:** The system is designed with OWASP Top 10 vulnerabilities in mind. Each vulnerability is addressed with a specific solution.
- **Efficient resource utilization:** Go as the main language of the system delivers small binary files, is memory efficient, and has a fast startup. Combined with a type-safe template engine (Templ), a server-side rendering (SSR), and a lightweight interactivity framework (HTMX).
- **Scalability and Maintainability:** The system is designed with the Hexagonal Architecture design pattern, which is a scalable and maintainable solution.

5.2. Future Work

1. Proctoring and Anti-Cheating Mechanisms
2. Advanced Question Types
3. Mobile-Responsive Application



Chapter VI

Appendices



6.1. Appendix A: Source Code

6.1.1. JWT Authentication Middleware

```
1 // internal/adapters/inbound/http/middleware/  
  auth_middleware.go  
2  
3 func (auth *AuthMiddleware) AuthenticationMiddleware()  
  gin.HandlerFunc {  
4     return func(ctx *gin.Context) {  
5         authHeader, err := ctx.Cookie("auth_token")  
6         if err != nil || authHeader == "" {  
7             handleUnauthorized(ctx)  
8             return  
9         }  
10  
11        claims, err := auth.jwt.ValidateToken(authHeader)  
12        if err != nil {  
13            auth.logger.LogErrorWithLevel("error",  
14                "HTTP_SERVER_ERROR",  
15                "TOKEN_VALIDATION_ERROR", "Token validation  
16                failed", err)  
17            ctx.SetCookie("auth_token", "", -1, "/", "",  
18                false, true)  
19            handleUnauthorized(ctx)  
20            return  
21        }  
22  
23        if claims != nil && claims.ExpiresAt <  
24        time.Now().Unix() {  
25            ctx.SetCookie("auth_token", "", -1, "/", "",  
26                false, true)  
27            handleUnauthorized(ctx)  
28            return  
29        }  
30  
31        setClaimsInContext(ctx, claims)  
32        ctx.Next()  
33    }  
34 }
```



```
28     }
29 }
```

6.1.2. Graceful Shutdown

```
1  func (s *Server) startWithGracefulShutdown(certFile,
   keyFile string) {
2      errChan := make(chan error, 1)
3      go func() {
4          var err error
5
6          // check if certFile and keyFile are provided
7          if certFile != "" && keyFile != "" {
8              err = s.server.ListenAndServeTLS(certFile, keyFile)
9          } else {
10             s.logger.LogErrorWithLevel("warn",
11                 logger.FailedToStartWithTLS.Type,
12                 logger.FailedToStartWithTLS.Code,
13                 logger.FailedToStartWithTLS.Msg, errors.New("unable to
14                     get TLS cert"))
15             err = s.server.ListenAndServe()
16         }
17         if err != nil && err != http.ErrServerClosed {
18             errChan <- fmt.Errorf("failed to start HTTP server: %v",
19                 err)
20         }
21     }()
22     s.waitForShutdownSignal(errChan)
23 }
```



6.1.3. Application Transaction Pattern

```
1  // Example: CreateExam in internal/application/
   exam_application.go
2
3  func (app *Application) CreateExam(ctx context.Context,
4      arg ports.CreateExamParams) (domain.Exam, error) {
5      var exam domain.Exam
```



```
6     err := app.txManager.WithTransaction(ctx, func(txCtx
    context.Context) error {
7         var err error
8         exam, err = app.examRepo.Create(txCtx, arg)
9         return err
10    })
11    return exam, err
12 }
```

6.1.4. Dependency Injection Pattern (func main)

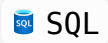
```
1  // cmd/api/main.go
2  func main() {
3      cfg, err := configInstance.GetConfig()
4      if err != nil {
5          log.Fatalf("Failed to get configuration: %v", err)
6      }
7
8      // Initialize Zap
9      zapLogger, err := logger.InitZapLogger(cfg)
10     if err != nil {
11         log.Fatalf("Failed to initialize logger: %v", err)
12     }
13     defer func() {
14         err := zapLogger.Sync()
15         if err != nil && !errors.Is(err, syscall.EINVAL) {
16             fmt.Fprintf(os.Stderr, "Failed to sync logger: %v\n",
17                 err)
18         }
19     }()
20
21     // Initialize Logger
22     zlog := logger.New(zapLogger)
23
24     DBConfig := &database.DBConfig{
25         // mapping from YAML config to DBConfig struct
26     }
```



```
27 pool, err := database.NewConnection(*DBConfig)
28 if err != nil {
29     zlog.LogErrorWithLevel("fatal", logger.DatabaseError,
30         logger.MongoFailedToConnect.Code, "Database failed to
31         connect", err)
32     return
33 }
34 query := sqlc.New(pool)
35 userRepo := repository.NewUserRepository(query)
36 subjectRepo := repository.NewSubjectRepository(query)
37 examRepo := repository.NewExamRepository(query)
38 questionRepo := repository.NewQuestionRepository(query)
39 txManager := database.NewPostgresTxManager(pool.Pool)
40 localDisk := storage.NewLocalDiskAdapter()
41 app := application.NewApplication(userRepo, subjectRepo,
42     examRepo, questionRepo, localDisk, txManager, pool, zlog)
43
44 // init validator
45 validate := validator.New()
46 // init jwt
47 jwt := helpers.NewJWT(cfg)
48 // init auth middleware
49 authMiddleware := middleware.NewAuthMiddleware(jwt, zlog)
50 // pass cache, repo, validator, jwt to controllers
51 userCtrl := handler.NewUserHandler(app, validate, jwt, cfg,
52     zlog)
53
54 // initialize the server
55 srv := server.NewServer(userCtrl, authMiddleware, zlog, cfg)
56
57 zlog.LogInfo(logger.ServerStartOK.Type,
58     logger.ServerStartOK.Msg, zap.String("server_address",
59     net.JoinHostPort(cfg.HTTPServer.Addr, cfg.HTTPServer.Port)))
60
61 // start the server over TLS
62 srv.StartOverTLS(cfg)
63 }
```

6.1.5. Database functions

```
1  -- Function to check if the number of choices is less
   than 4
2  CREATE OR REPLACE FUNCTION check_choices_count()
3  RETURNS TRIGGER AS $$
4  DECLARE
5      choice_count INT;
6  BEGIN
7      SELECT COUNT(*) INTO choice_count
8      FROM choices
9      WHERE question_id = NEW.question_id;
10
11     IF choice_count >= 4 THEN
12         RAISE EXCEPTION 'Maximum 4 choices allowed per
13         question';
14     END IF;
15     RETURN NEW;
16 $$ LANGUAGE plpgsql;
```



6.2. Appendix B: User Interface Screens

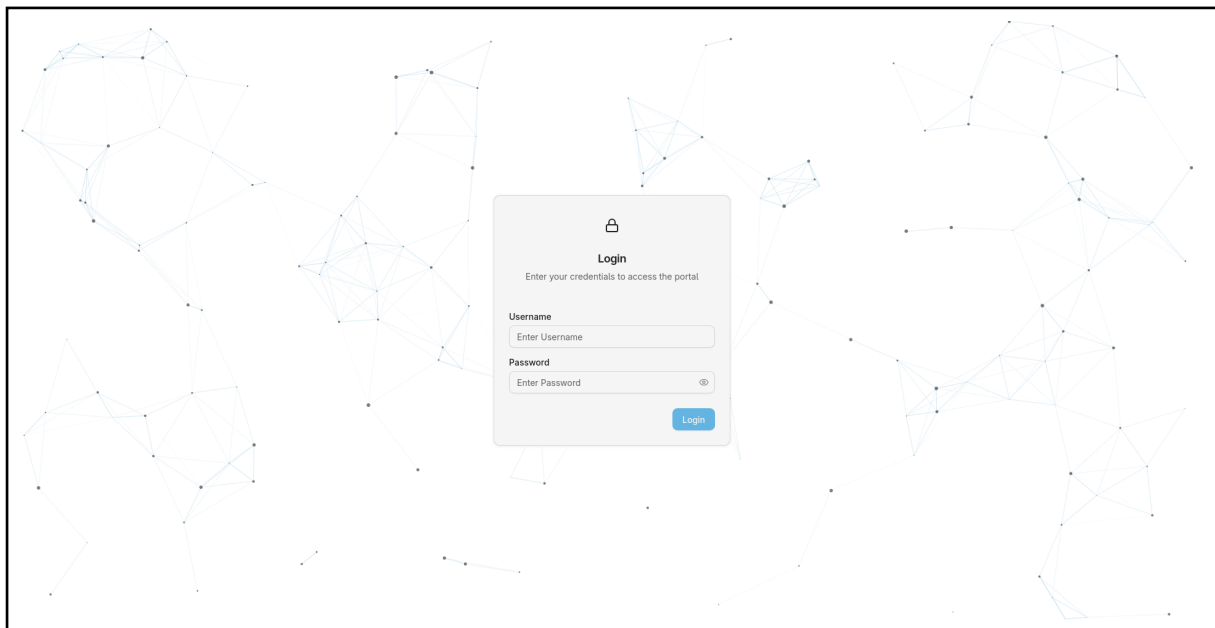


Figure 24: Login Page

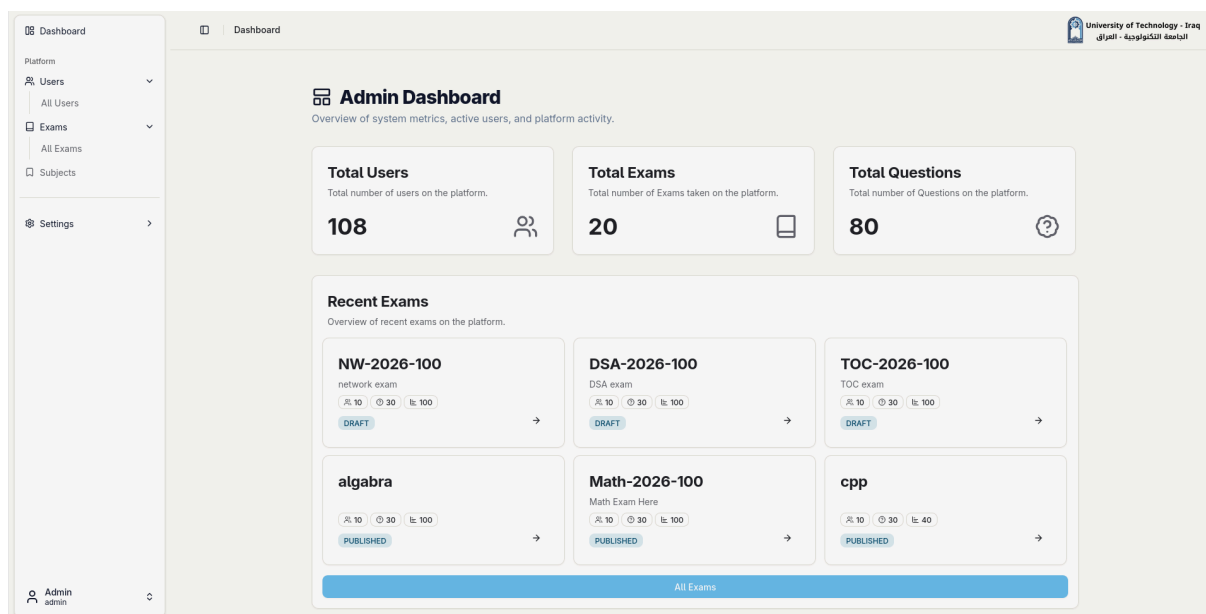


Figure 25: Admin Dashboard Page

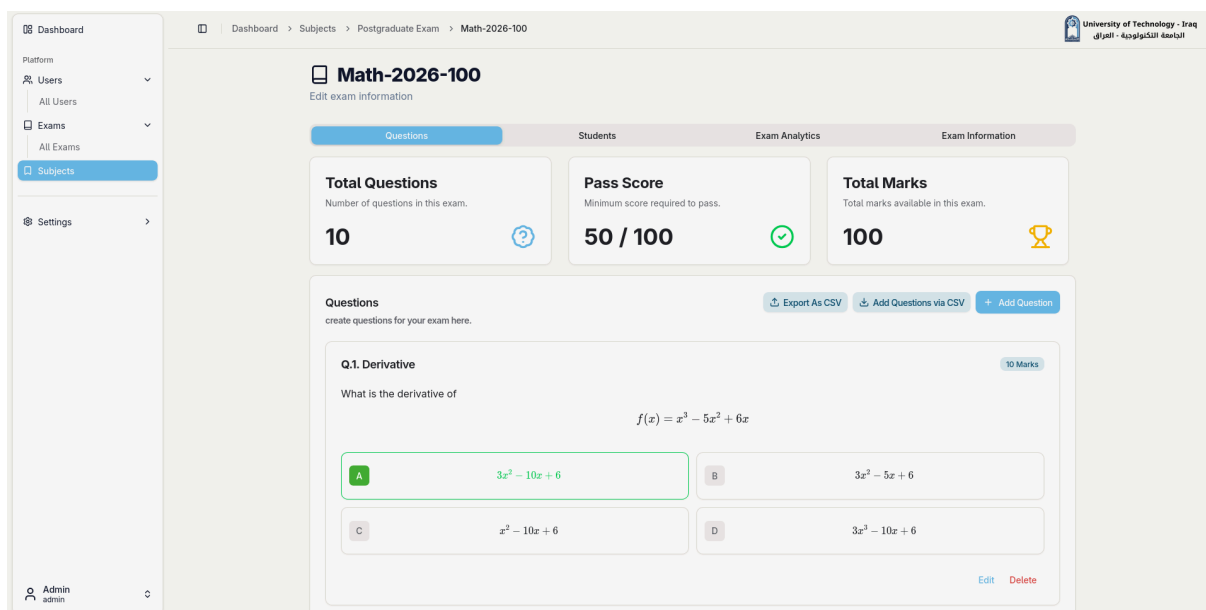


Figure 26: Edit Exam Page

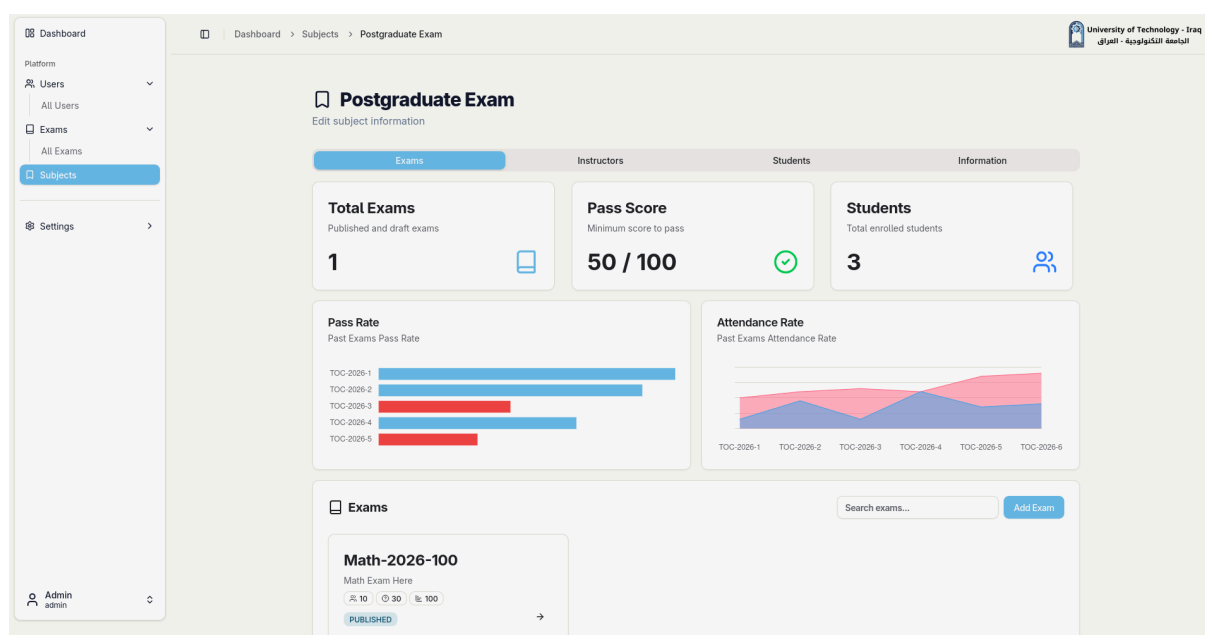


Figure 27: Edit Subject Dashboard Page

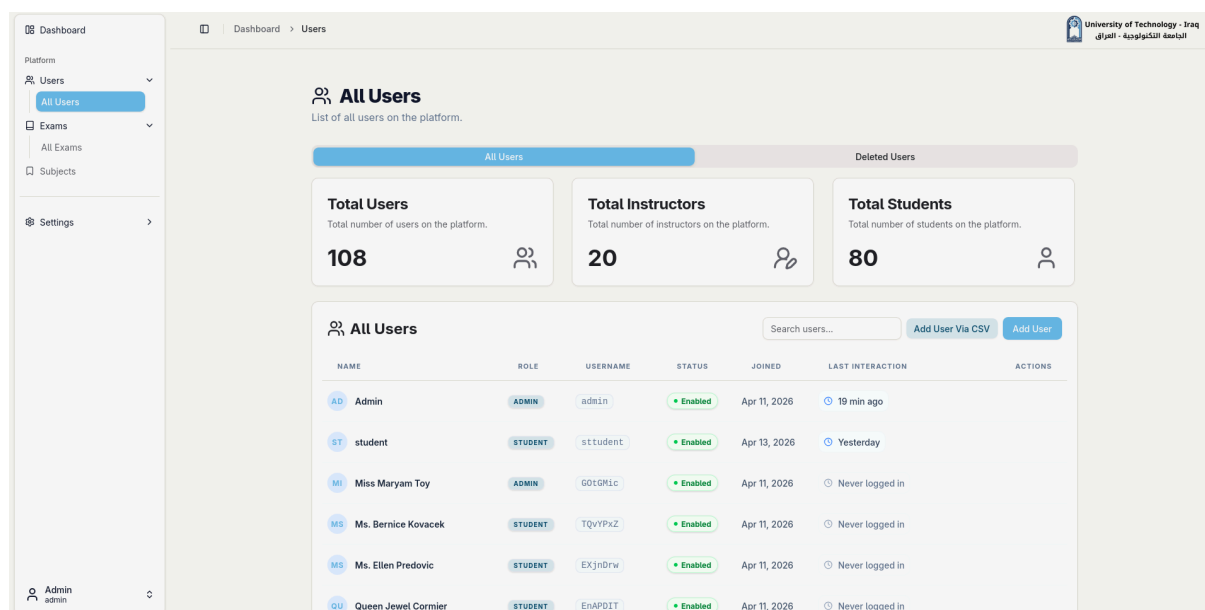


Figure 28: All Users Page

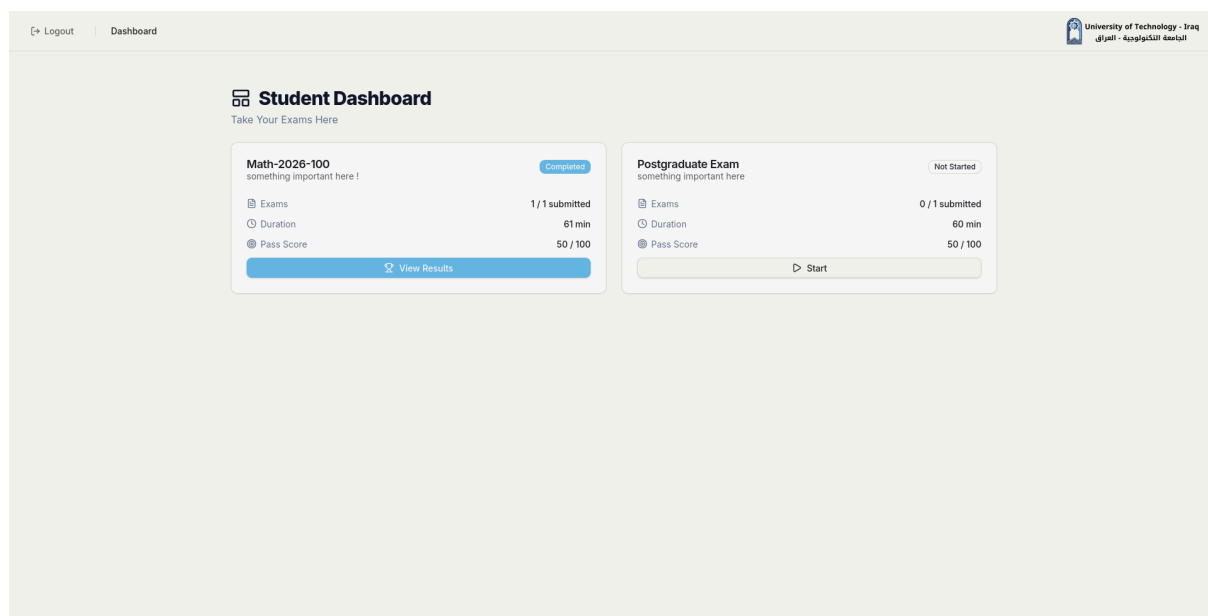


Figure 29: Student Dashboard Page

Bibliography

- [1] V. Kumar, B. Kumar, and N. Molakatala, "E-PARIKSHA (Online Exam Portal) for skill Development of Students A Project Report Submitted in partial fulfillment of the degree of Master of Computer Application," 2022.
- [2] S. Pokhrel, S. Gurung, R. Chaudhary, and S. Jhingran, "A Secure Online Examination Platform: Integrating AI-Based Proctoring and Full Stack Architecture for Enhanced Academic Integrity," *International Conference on Future Technologies (ICFT)*, 2025.
- [3] R. R. Subramanian, B. Surekha, P. SubbaRao, G. Chowdary, and M. Venkatesh, "EduGuard – AI Based Online Exam Management System," 2025, pp. 1–6. doi: 10.1109/ICCRTEE64519.2025.11052922.
- [4] L. Pisu, D. Maiorca, and G. Giacinto, "An Assessment of the Overlooked Dangers of Template Engines," *ACM Transactions on the Web*, Feb. 2026, doi: 10.1145/3799796.
- [5] A. Hoenig, "Templ: A HTML Templating Language for Go." [Online]. Available: <https://github.com/a-h/templ>
- [6] CVE Program, "CVE-2025-55182: Pre-authentication remote code execution vulnerability in React Server Components." [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2025-55182>
- [7] CVE Program, "CVE-2025-66412: Stored Cross-Site Scripting (XSS) vulnerability in Angular Template Compiler via SVG/MathML." [Online]. Available: <https://www.cve.org/CVERecord?id=CVE-2025-66412>
- [8] K. S. Jasmine and V. S. Sridhara, "Normalizing a database: A case study," *International Journal of Computer Science and Network Security (IJCSNS)*, vol. 10, no. 8, pp. 282–288, 2010.
- [9] K. Gray, "sqlc." [Online]. Available: <https://github.com/sqlc-dev/sqlc>
- [10] A. Cockburn, "Hexagonal architecture." [Online]. Available: <https://alistair.cockburn.us/hexagonal-architecture>

-
- [11] M. Noback, “The Dependency Inversion Principle,” in *Principles of Package Design: Creating Reusable Software Components*, Berkeley, CA: Apress, 2018, pp. 67–104. doi: 10.1007/978-1-4842-4119-6_5.
 - [12] OWASP Foundation, “OWASP Top 10:2025.” [Online]. Available: <https://owasp.org/Top10/2025/>
 - [13] J. Hu, W. Zhao, and Y. Cui, “A Survey on SQL Injection Attacks, Detection and Prevention,” in *Proceedings of the 2020 12th International Conference on Machine Learning and Computing*, ACM, Feb. 2020, pp. 483–488. doi: 10.1145/3383972.3384028.

الخلاصة

من خلال استخدام النظام الإلكتروني لإدارة الامتحانات الأكاديمية، يمكن للمشرفين وأعضاء الهيئة التدريسية والطلاب الاستفادة من مزايا عديدة. فأولاً، يتيح النظام توفير الوقت والجهد المبذول في العمليات اليدوية والورقية التقليدية، حيث يمكن الآن تنفيذ المهام الإدارية وتصحيح الامتحانات بشكل أكثر فاعلية وسهولة.

ثانياً، يعمل النظام على تحسين تنظيم وتخزين بيانات الامتحانات والنتائج، مما يسهل استرجاع المعلومات وتحليلها بسرعة ودقة عالية. لم يعد هناك حاجة للبحث اليدوي في السجلات الورقية، بل يمكن للمشرفين الوصول إلى كافة البيانات المتعلقة بالطلاب والاختبارات بنقرة واحدة فقط مع ضمان أمنها ونزاهتها.

ثالثاً، يساهم النظام في تحسين التواصل والشفافية بين الجميع، حيث يوفر إشعارات فورية ونتائج تلقائية دقيقة. يمكن للطلاب الحصول على تحديثات مستمرة حول امتحاناتهم ونتائجهم وتحميل تقارير الأداء بصيغة PDF، مما يساعدهم على متابعة تقدمهم الأكاديمي والتفاعل مع المواد الدراسية بشكل أفضل.

وفي النهاية، يمكن القول إن نظام إدارة الامتحانات هذا يوفر تحسينات جوهرية وفوائد ملموسة لكافة الأطراف المعنية. إنه يساعد في تبسيط العمليات الأكاديمية، وتعزيز أمن البيانات، وتوليد التقارير في الوقت الحقيقي، مما يساهم في إدارة الكليات والمؤسسات التعليمية بشكل فعال ومؤتمت بالكامل.



وزارة التعليم العالي والبحث العلمي
الجامعة التكنولوجية
كلية هندسة الحاسوب
قسم هندسة الحاسوب



تصميم وتنفيذ نظام امتحانات فعال و امن

مشروع مقدم إلى كلية هندسة الحاسوب في الجامعة التكنولوجية كجزء من
متطلبات نيل درجة البكالوريوس في هندسة الحاسوب

بإشراف

أ.د. ضاري عادل محمود

إعداد الطلبة

الحسن عدي صاحب

نور حسن عبد المهدي

2025-2026